

KTH Royal Institute of Technology

Omogen Heap

Simon Lindholm, Johan Sannemo, Mårten Wiman

2026-10-08

1 竞赛工具 1

2 数学 1

3 数据结构 3

4 数值方法 5

5 数论 10

6 组合数学 12

7 图论 13

8 计算几何 20

9 字符串 24

10 杂项 26

竞赛工具 (1)

template.cpp 20 lines

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'

using ll = long long;
using ld = long double;
using ull = unsigned long long;
using pll = pair<long long, long long>;
using pld = pair<long double, long double>;
using pii = pair<int, int>;
using vi = vector<int>;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);

    return 0;
}
```

.bashrc 3 lines

```
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 \
  -fsanitize=undefined,address'
xmodmap -e 'clear lock' -e 'keycode 66=less greater' # 大写锁定键 → <>
```

.vimrc 6 lines

```
set cin aw ai is ts=4 sw=4 tm=50 nu noeb bg=dark ru cul
sy on   |   im jk <esc>   |   im kj <esc>   |   no ; :
" 选中一段文本后输入 :Hash,即可计算选区的哈希。
" 用于核对誊抄是否有误。
ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \
 \| md5sum \| cut -c-6
```

hash.sh 2 lines

```
# 计算文件哈希,忽略所有空白与注释。用于核对代码誊抄是否正确。
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -c-6
```

troubleshoot.txt 52 lines

```
提交前:
如果样例不够,自己写几个简单测试。
时间限制接近吗?接近的话生成最大数据。
内存占用是否正常?
有没有可能溢出的地方?
确认提交的是正确的文件。

答案错误:
打印你的解法!连同调试输出一起打印。
换了测试点后有没有清空所有数据结构?
算法能否处理输入的全部范围?
重读一遍完整题面。
所有边界情况都处理对了吗?
题目理解是否正确?
有没有未初始化的变量?
有没有溢出?
弄混了 N 和 M、i 和 j 之类的?
确定你的算法对吗?
有哪些没想到的特殊情况?
你用的 STL 函数行为确定和你想的一致吗?
加一些断言,也许重新提交一次。
造一些测试数据来跑你的算法。
用一个简单例子走一遍算法。
再把这份清单过一遍。
向队友解释你的算法。
请队友看看你的代码。
出去走一小圈,比如去趟洗手间。
输出格式对吗?(包括空白字符)
重写一遍解法,或让队友重写。

运行时错误:
所有边界情况都在本地测过吗?
有没有未初始化的变量?
有没有越界读写任何 vector?
有没有可能失败的断言?
有没有可能除以 0?(比如模 0)
有没有可能的无限递归?
指针或迭代器失效?
内存是不是用多了?
用重新提交来调试(例如重映射信号,见"杂项"章)。

超时:
有没有可能的死循环?
算法的复杂度是多少?
有没有复制大量不必要的数据?(改用引用)
输入输出有多大?(考虑 scanf)
避免 vector、map。(用数组/unordered_map)
队友怎么看你的算法?

内存超限:
算法最多需要多少内存?
换了测试点后有没有清空所有数据结构?
```

数学 (2)

2.1 方程

$$ax^2+bx+c=0 \Rightarrow x=\frac{-b\pm\sqrt{b^2-4ac}}{2a}$$

极值点为 $x=-b/2a$。

$$\begin{matrix} ax+by=e \\ cx+dy=f \end{matrix} \Rightarrow \begin{matrix} x=\dfrac{ed-bf}{ad-bc} \\ y=\dfrac{af-ec}{ad-bc} \end{matrix}$$

一般地, 给定方程 $Ax=b$, 变量 x_i 的解为

$$x_i=\frac{\det A_i'}{\det A}$$

其中 A_i' 是把 A 的第 i 列换成 b 所得的矩阵。

2.2 递推

若 $a_n=c_1a_{n-1}+\cdots+c_ka_{n-k}$,$r_1,\ldots,r_k$ 是 $x^k-c_1x^{k-1}-\cdots-c_k$ 的互异根, 则存在 $d_1,\ldots,d_k$ 使

$$a_n=d_1r_1^n+\cdots+d_kr_k^n.$$

重根 r 变为多项式因子, 如 $a_n=(d_1n+d_2)r^n$。

2.3 三角

$$\sin(v+w)=\sin v\cos w+\cos v\sin w$$
$$\cos(v+w)=\cos v\cos w-\sin v\sin w$$

$$\tan(v+w)=\frac{\tan v+\tan w}{1-\tan v\tan w}$$
$$\sin v+\sin w=2\sin\frac{v+w}{2}\cos\frac{v-w}{2}$$
$$\cos v+\cos w=2\cos\frac{v+w}{2}\cos\frac{v-w}{2}$$

$$(V+W)\tan(v-w)/2=(V-W)\tan(v+w)/2$$

其中 V,W 为角 v,w 所对边长。

$$a\cos x+b\sin x=r\cos(x-\phi)$$
$$a\sin x+b\cos x=r\sin(x+\phi)$$

其中 $r=\sqrt{a^2+b^2},\phi=\operatorname{atan2}(b,a)$。

2.4 几何

2.4.1 三角形

边长:a,b,c

半周长:$p=\dfrac{a+b+c}{2}$

面积:$A=\sqrt{p(p-a)(p-b)(p-c)}$

外接圆半径:$R=\dfrac{abc}{4A}$

内切圆半径:$r=\dfrac{A}{p}$

中线长 (把三角形分成两个等面积三角形):$m_a = \frac{1}{2}\sqrt{2b^2+2c^2-a^2}$

角平分线长 (把角平分):$s_a = \sqrt{bc\left[1-\left(\frac{a}{b+c}\right)^2\right]}$

正弦定理:$\frac{\sin\alpha}{a} = \frac{\sin\beta}{b} = \frac{\sin\gamma}{c} = \frac{1}{2R}$

余弦定理:$a^2 = b^2+c^2-2bc\cos\alpha$

正切定理:$\frac{a+b}{a-b} = \frac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

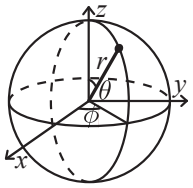
2.4.2 四边形

边长 a,b,c,d, 对角线 e,f, 对角线夹角 θ, 面积 A, 以及魔法量 $F = b^2+d^2-a^2-c^2$:

$$4A = 2ef\cdot\sin\theta = F\tan\theta = \sqrt{4e^2f^2-F^2}$$

对圆内接四边形, 对角之和为 $180°$, $ef = ac+bd$, 且 $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$。

2.4.3 球坐标



$$\begin{array}{cc} x = r\sin\theta\cos\phi & r = \sqrt{x^2+y^2+z^2} \\ y = r\sin\theta\sin\phi & \theta = \operatorname{acos}(z/\sqrt{x^2+y^2+z^2}) \\ z = r\cos\theta & \phi = \operatorname{atan2}(y,x) \end{array}$$

2.5 导数与积分

$$\begin{array}{cc} \frac{d}{dx}\arcsin x = \frac{1}{\sqrt{1-x^2}} & \frac{d}{dx}\arccos x = -\frac{1}{\sqrt{1-x^2}} \\ \frac{d}{dx}\tan x = 1+\tan^2 x & \frac{d}{dx}\arctan x = \frac{1}{1+x^2} \\ \int\tan ax = -\frac{\ln|\cos ax|}{a} & \int x\sin ax = \frac{\sin ax - ax\cos ax}{a^2} \\ \int e^{-x^2} = \frac{\sqrt{\pi}}{2}\operatorname{erf}(x) & \int xe^{ax}dx = \frac{e^{ax}}{a^2}(ax-1) \end{array}$$

分部积分:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 求和

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1}-c^a}{c-1}, c\neq 1$$

$$1+2+3+\cdots+n = \frac{n(n+1)}{2}$$

$$1^2+2^2+3^2+\cdots+n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3+2^3+3^3+\cdots+n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4+2^4+3^4+\cdots+n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.7 级数

$$e^x = 1+x+\frac{x^2}{2!}+\frac{x^3}{3!}+\ldots,\ (-\infty<x<\infty)$$

$$\ln(1+x) = x-\frac{x^2}{2}+\frac{x^3}{3}-\frac{x^4}{4}+\ldots,\ (-1<x\le 1)$$

$$\sqrt{1+x} = 1+\frac{x}{2}-\frac{x^2}{8}+\frac{2x^3}{32}-\frac{5x^4}{128}+\ldots,\ (-1\le x\le 1)$$

$$\sin x = x-\frac{x^3}{3!}+\frac{x^5}{5!}-\frac{x^7}{7!}+\ldots,\ (-\infty<x<\infty)$$

$$\cos x = 1-\frac{x^2}{2!}+\frac{x^4}{4!}-\frac{x^6}{6!}+\ldots,\ (-\infty<x<\infty)$$

2.8 概率论

设 X 为离散随机变量, 取值 x 的概率为 $p_X(x)$。则其期望 (均值) 为 $\mu = \mathbb{E}(X) = \sum_x xp_X(x)$, 方差为 $\sigma^2 = V(X) = \mathbb{E}(X^2)-(\mathbb{E}(X))^2 = \sum_x(x-\mathbb{E}(X))^2p_X(x)$, 其中 σ 为标准差。若 X 是连续的, 则有概率密度函数 $f_X(x)$, 上面的求和相应变为积分, 并把 $p_X(x)$ 换成 $f_X(x)$。

期望是线性的:

$$\mathbb{E}(aX+bY) = a\mathbb{E}(X)+b\mathbb{E}(Y)$$

对独立的 X 与 Y,

$$V(aX+bY) = a^2V(X)+b^2V(Y).$$

2.8.1 离散分布

二项分布

n 次独立的是/否实验中, 每次成功概率为 p, 成功次数服从 $\mathrm{Bin}(n,p)$, $n=1,2,\ldots$, $0\le p\le 1$。

$$p(k) = \binom{n}{k}p^k(1-p)^{n-k}$$

$$\mu = np,\ \sigma^2 = np(1-p)$$

p 较小时,$\mathrm{Bin}(n,p)$ 近似为 $\mathrm{Po}(np)$。

首次成功分布

独立的是/否实验中, 首次成功所需的实验次数服从 $\mathrm{Fs}(p)$, $0\le p\le 1$。

$$p(k) = p(1-p)^{k-1},\ k=1,2,\ldots$$

$$\mu = \frac{1}{p},\ \sigma^2 = \frac{1-p}{p^2}$$

泊松分布

固定时间 t 内发生的事件数, 若事件以已知平均速率 κ 发生且与上次事件后经过的时间无关, 则服从 $\mathrm{Po}(\lambda)$, $\lambda = t\kappa$。

$$p(k) = e^{-\lambda}\frac{\lambda^k}{k!}, k=0,1,2,\ldots$$

$$\mu = \lambda,\ \sigma^2 = \lambda$$

2.8.2 连续分布

均匀分布

概率密度函数在 a 与 b 之间为常数、其余为 0 时, 服从 $\mathrm{U}(a,b)$, $a<b$。

$$f(x) = \begin{cases} \frac{1}{b-a} & a<x<b \\ 0 & \text{其它} \end{cases}$$

$$\mu = \frac{a+b}{2},\ \sigma^2 = \frac{(b-a)^2}{12}$$

指数分布

泊松过程中两次事件之间的时间服从 $\mathrm{Exp}(\lambda)$, $\lambda>0$。

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x\ge 0 \\ 0 & x<0 \end{cases}$$

$$\mu = \frac{1}{\lambda},\ \sigma^2 = \frac{1}{\lambda^2}$$

正态分布

大多数均值为 μ、方差为 σ^2 的实际随机值都可由 $\mathcal{N}(\mu,\sigma^2)$, $\sigma>0$ 很好地描述。

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

若 $X_1\sim\mathcal{N}(\mu_1,\sigma_1^2)$ 且 $X_2\sim\mathcal{N}(\mu_2,\sigma_2^2)$, 则

$$aX_1+bX_2+c\sim\mathcal{N}(\mu_1+\mu_2+c, a^2\sigma_1^2+b^2\sigma_2^2)$$

2.9 马尔可夫链

马尔可夫链是一种离散随机过程, 其性质是下一状态只依赖当前状态。设 $X_1, X_2, \ldots$ 为马尔可夫过程生成的随机变量序列。则存在转移矩阵 $\mathbf{P} = (p_{ij}), p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, 且 $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ 是 X_n 的概率分布 (即 $p_i^{(n)} = \Pr(X_n = i)$), 其中 $\mathbf{p}^{(0)}$ 是初始分布。

π 是平稳分布, 若 $\pi = \pi\mathbf{P}$。若马尔可夫链不可约 (从任意状态可到达任意状态), 则 $\pi_i = \frac{1}{\mathbb{E}(T_i)}$, 其中 $\mathbb{E}(T_i)$ 是两次访问状态 i 之间的期望时间。π_j / π_i 是两次访问状态 i 之间访问状态 j 的期望次数。

对连通、无向、非二分图, 转移概率在所有邻居间均匀时,π_i 正比于节点 i 的度数。

马尔可夫链是遍历的, 若其渐近分布与初始分布无关。有限马尔可夫链遍历当且仅当其不可约且非周期 (即所有环长的 gcd 为 1)。$\lim_{k\to\infty} \mathbf{P}^k = \mathbf{1}\pi$。

马尔可夫链是 A-链, 若状态可分为 $\mathbf{A}$ 与 $\mathbf{G}$ 两组, 使 $\mathbf{A}$ 中所有状态都是吸收态 ($p_{ii} = 1$), 且 $\mathbf{G}$ 中所有状态都通向 $\mathbf{A}$ 中的吸收态。初始状态为 j 时, 在状态 $i \in \mathbf{A}$ 被吸收的概率为 $a_{ij} = p_{ij} + \sum_{k\in\mathbf{G}} a_{ik} p_{kj}$。初始状态为 i 时, 到吸收的期望时间为 $t_i = 1 + \sum_{k\in\mathbf{G}} p_{ki} t_k$。

数据结构 (3)

OrderStatisticTree.h

描述: 一个集合 (非多重集!), 支持查询第 n 小元素以及某元素的下标 (排名)。想要 map 版本, 请修改 null_type。
时间: $\mathcal{O}(\log N)$

782797, 18 lines

```
#include <bits/extc++.h>
using namespace __gnu_pbds;

template <class T>
using Tree = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

void example() {
    Tree<int> t, t2;
    t.insert(8);

    auto it = t.insert(10).first;
    assert(it == t.lower_bound(9));
    assert(t.order_of_key(10) == 1);
    assert(t.order_of_key(11) == 2);
    assert(*t.find_by_order(0) == 8);
    t.join(t2); // 前提是 T < T2 或 T > T2,把 t2 并入 t
}
```

HashMap.h

描述: 与 unordered_map 接口基本一致的哈希表, 速度约为其 3 倍, 内存占用为其 1.5 倍。初始容量必须为 2 的幂 (若提供)。

d77092, 7 lines

```
#include <bits/extc++.h>
// 为了尽量利用高位而不是低位:
struct chash { // C 取大的奇数
    const uint64_t C = ll(4e18 * acos(0)) | 71;
    ll operator()(ll x) const { return __builtin_bswap64(x * C)
        ; }
};
__gnu_pbds::gp_hash_table<ll, int, chash> h({}, {}, {}, {}, {1
    << 16});
```

SegmentTree.h

描述: 零下标最大线段树, 区间左闭右开。可通过修改 T、f 和单位元来改变用途。
时间: $\mathcal{O}(\log N)$

8b0515, 27 lines

```
struct Tree {
    using T = int;
    static constexpr T unit = INT_MIN;

    T f(T a, T b) { return max(a, b); } // 任意结合运算
    vector<T> s;
    int n;

    Tree(int n = 0, T def = unit) : s(2 * n, def), n(n) {}

    void update(int pos, T val) {
        for (s[pos += n] = val; pos /= 2;) {
            s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
        }
    }

    T query(int b, int e) { // 查询区间 [b, e)
        T ra = unit, rb = unit;

        for (b += n, e += n; b < e; b /= 2, e /= 2) {
            if (b % 2) ra = f(ra, s[b++]);
            if (e % 2) rb = f(s[--e], rb);
        }

        return f(ra, rb);
    }
};
```

LazySegmentTree.h

描述: 支持大区间加/赋值并计算区间最大值的线段树 (push 式动态开点)。可以改成其它运算。配合 bump allocator 使用性能更好; 用 SmallPtr 或隐式下标可节省内存。
用法: Node* tr = new Node(v, 0, sz(v));
时间: $\mathcal{O}(\log N)$。

"../various/BumpAllocator.h" f3e663, 66 lines

```
const int inf = 1e9;
struct Node {
    Node *l = 0, *r = 0;
    int lo, hi, mset = inf, madd = 0, val = -inf;

    Node(int lo, int hi) : lo(lo), hi(hi) {} // 一大段 -inf 区间
    Node(vi &v, int lo, int hi) : lo(lo), hi(hi) {
        if (lo + 1 < hi) {
            int mid = lo + (hi - lo) / 2;
            l = new Node(v, lo, mid);
            r = new Node(v, mid, hi);
            val = max(l->val, r->val);
        } else val = v[lo];
    }

    int query(int L, int R) {
        if (R <= lo || hi <= L) return -inf;
        if (L <= lo && hi <= R) return val;
        push();
        return max(l->query(L, R), r->query(L, R));
    }

    void set(int L, int R, int x) {
        if (R <= lo || hi <= L) return;
        if (L <= lo && hi <= R) {
            mset = val = x;
            madd = 0;
        } else {
            push();
            l->set(L, R, x);
            r->set(L, R, x);
            val = max(l->val, r->val);
        }
    }

    void add(int L, int R, int x) {
        if (R <= lo || hi <= L) return;
        if (L <= lo && hi <= R) {
            if (mset != inf) mset += x;
            else madd += x;
            val += x;
        } else {
            push();
            l->add(L, R, x);
            r->add(L, R, x);
            val = max(l->val, r->val);
        }
    }

    void push() {
        if (!l) {
            int mid = lo + (hi - lo) / 2;
            l = new Node(lo, mid);
            r = new Node(mid, hi);
        }
        if (mset != inf) {
            l->set(lo, hi, mset);
            r->set(lo, hi, mset);
            mset = inf;
        } else if (madd) {
            l->add(lo, hi, madd);
            r->add(lo, hi, madd);
            madd = 0;
        }
    }
};
```

UnionFindRollback.h

描述: 支持撤销 (回滚) 的并查集。若不需要撤销, 可忽略 st、time() 与 rollback()。
用法: int t = uf.time(); ...; uf.rollback(t);
时间: $\mathcal{O}(\log(N))$

6fb32e, 29 lines

```
struct RollbackUF {
    vi e;
    vector<pii> st;

    RollbackUF(int n) : e(n, -1) {}

    int size(int x) { return -e[find(x)]; }
    int find(int x) { return e[x] < 0 ? x : find(e[x]); }
    int time() { return (int)st.size(); }

    void rollback(int t) {
        for (int i = time() - 1; i >= t; i--) {
            e[st[i].first] = st[i].second;
        }
        st.resize(t);
    }

    bool join(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
```

```
        st.push_back({a, e[a]});
        st.push_back({b, e[b]});
        e[a] += e[b];
        e[b] = a;
        return true;
    }
};
```

SubMatrix.h

描述: 快速计算子矩阵和, 给定左上角与右下角 (左闭右开)。
用法: SubMatrix<int> m(matrix);
m.sum(0, 0, 2, 2); // 左上角 2x2 的 4 个元素
时间: $\mathcal{O}\left(N^2+Q\right)$

c027fc, 19 lines

```
template <class T>
struct SubMatrix {
    vector<vector<T>> p;

    SubMatrix(vector<vector<T>> &v) {
        int R = (int)v.size(), C = (int)v[0].size();
        p.assign(R + 1, vector<T>(C + 1));

        for (int r = 0; r < R; r++) {
            for (int c = 0; c < C; c++) {
                p[r + 1][c + 1] = v[r][c] + p[r][c + 1] + p[r +
                    1][c] - p[r][c];
            }
        }
    }

    T sum(int u, int l, int d, int r) {
        return p[d][r] - p[d][l] - p[u][r] + p[u][l];
    }
};
```

Matrix.h

描述: 方阵上的基本运算。
用法: Matrix<int, 3> A;
A.d = {{{{1,2,3}}, {{4,5,6}}, {{7,8,9}}}};
array<int, 3> vec = {1,2,3};
vec = (A^N) * vec;

5150f9, 48 lines

```
template <class T, int N>
struct Matrix {
    using M = Matrix;
    array<array<T, N>, N> d{};

    M operator*(const M &m) const {
        M a;

        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                for (int k = 0; k < N; k++) {
                    a.d[i][k] += d[i][j] * m.d[j][k];
                }
            }
        }

        return a;
    }

    array<T, N> operator*(const array<T, N> &vec) const {
        array<T, N> ret{};

        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                ret[i] += d[i][j] * vec[j];
            }
        }

        return ret;
    }

    M operator^(ll p) const {
        assert(p >= 0);
        M a, b(*this);

        for (int i = 0; i < N; i++) {
            a.d[i][i] = 1;
        }

        while (p) {
            if (p & 1) a = a * b;
            b = b * b;
            p >>= 1;
        }

        return a;
    }
};
```

LineContainer.h

描述: 一个容器, 可以加入形如 kx+m 的直线, 并查询某点 x 处的最大值。常用于动态规划 (凸包技巧,convex hull trick)。
时间: $\mathcal{O}\left(\log N\right)$

1d01b8, 40 lines

```
struct Line {
    mutable ll k, m, p;

    bool operator<(const Line &o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (若是 double,取 inf = 1/.0,div(a,b) = a/b)
    static const ll inf = LLONG_MAX;

    ll div(ll a, ll b) { // 向下取整的除法
        return a / b - ((a ^ b) < 0 && a % b);
    }

    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }

    void add(ll k, ll m) {
        auto z = insert({k, m, 0});
        auto y = z++;
        auto x = y;

        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y
            ));
        while ((y = x) != begin() && (--x)->p >= y->p) {
            isect(x, erase(y));
        }
    }

    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};
```

Treap.h

描述: 一种简短的自平衡树。可作为支持对数时间分裂/合并的顺序容器, 且易于附加额外数据。
时间: $\mathcal{O}\left(\log N\right)$

50fc84, 63 lines

```
struct Node {
    Node *l = 0, *r = 0;
    int val, y, c = 1;

    Node(int val) : val(val), y(rand()) {}
    void recalc();
};

int cnt(Node *n) { return n ? n->c : 0; }
void Node::recalc() { c = cnt(l) + cnt(r) + 1; }

template <class F>
void each(Node *n, F f) {
    if (n) {
        each(n->l, f);
        f(n->val);
        each(n->r, f);
    }
}

pair<Node *, Node *> split(Node *n, int k) {
    if (!n) return {};
    if (cnt(n->l) >= k) { // 若要实现 lower_bound(k),改成 "n->
        val >= k"
        auto [L, R] = split(n->l, k);
        n->l = R;
        n->recalc();
        return {L, n};
    } else {
        auto [L, R] = split(n->r, k - cnt(n->l) - 1); // 或者直
            接用 "k"
        n->r = L;
        n->recalc();
        return {n, R};
    }
}

Node *merge(Node *l, Node *r) {
    if (!l) return r;
    if (!r) return l;
    if (l->y > r->y) {
        l->r = merge(l->r, r);
        l->recalc();
        return l;
    } else {
        r->l = merge(l, r->l);
        r->recalc();
        return r;
    }
}

Node *ins(Node *t, Node *n, int pos) {
    auto [l, r] = split(t, pos);
    return merge(merge(l, n), r);
}

// 示例用法:把区间 [l, r) 移动到下标 k 处
void move(Node *&t, int l, int r, int k) {
    Node *a, *b, *c;

    tie(a, b) = split(t, l);
    tie(b, c) = split(b, r - l);
    if (k <= l) t = merge(ins(a, b, k), c);
    else t = merge(a, ins(c, b, k - r));
```

```
}
```

FenwickTree.h

描述: 计算前缀和 a[0] + a[1] + ... + a[pos - 1], 支持单点修改 (按新旧值之差更新)。
时间: 两个操作都是 $\mathcal{O}(\log N)$。

8f0595, 36 lines

```
struct FT {
    vector<ll> s;

    FT(int n) : s(n) {}

    void update(int pos, ll dif) { // a[pos] += dif
        for (; pos < (int)s.size(); pos |= pos + 1) {
            s[pos] += dif;
        }
    }

    ll query(int pos) { // [0, pos) 的元素和
        ll res = 0;

        for (; pos > 0; pos &= pos - 1) {
            res += s[pos - 1];
        }

        return res;
    }

    int lower_bound(ll sum) { // 最小的 pos 使 [0, pos] 的元素和 >= sum
        // 若不存在这样的前缀和返回 n;若 sum <= 0 返回 -1
        if (sum <= 0) return -1;
        int pos = 0;

        for (int pw = 1 << 25; pw; pw >>= 1) {
            if (pos + pw <= (int)s.size() && s[pos + pw - 1] < sum) {
                pos += pw;
                sum -= s[pos - 1];
            }
        }

        return pos;
    }
};
```

FenwickTree2d.h

描述: 计算 a[i,j](i<I, j<J) 的二维前缀和, 支持单点加。要求所有会被更新的位置事先已知 (在 init() 前先调用 fakeUpdate())。
时间: $\mathcal{O}(\log^2 N)$。(若要 $\mathcal{O}(\log N)$, 请使用可持久化线段树。)

"FenwickTree.h" a41fe1, 39 lines

```
struct FT2 {
    vector<vi> ys;
    vector<FT> ft;

    FT2(int limx) : ys(limx) {}

    void fakeUpdate(int x, int y) {
        for (; x < (int)ys.size(); x |= x + 1) {
            ys[x].push_back(y);
        }
    }

    void init() {
        for (vi &v : ys) {
            sort(v.begin(), v.end());
            ft.emplace_back((int)v.size());
        }
    }

    int ind(int x, int y) {
        return (int)(lower_bound(ys[x].begin(), ys[x].end(), y)
                - ys[x].begin());
    }

    void update(int x, int y, ll dif) {
        for (; x < (int)ys.size(); x |= x + 1) {
            ft[x].update(ind(x, y), dif);
        }
    }

    ll query(int x, int y) {
        ll sum = 0;

        for (; x; x &= x - 1) {
            sum += ft[x - 1].query(ind(x - 1, y));
        }

        return sum;
    }
};
```

RMQ.h

描述: 数组上的静态 RMQ(区间最小值查询)。O(1) 返回 min(V[a], V[a + 1], ... V[b - 1])。
用法: RMQ rmq(values);
rmq.query(inclusive, exclusive);
时间: $\mathcal{O}(|V| \log |V| + Q)$

72379d, 20 lines

```
template <class T>
struct RMQ {
    vector<vector<T>> jmp;

    RMQ(const vector<T> &V) : jmp(1, V) {
        for (int pw = 1, k = 1; pw * 2 <= (int)V.size(); pw *= 2, k++) {
            jmp.emplace_back((int)V.size() - pw * 2 + 1);

            for (int j = 0; j < (int)jmp[k].size(); j++) {
                jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
            }
        }
    }

    T query(int a, int b) {
        assert(a < b); // a == b 时也可改为返回 inf
        int dep = 31 - __builtin_clz(b - a);
        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};
```

MoQueries.h

描述: 把询问看作点, 求一条近似 TSP 路线的顺序来处理区间 (或树上路径) 询问, 相邻询问之间通过在端点处增删元素来转移。若值在树边上, 把 step 改为增删边 (a, c), 并去掉开头的 add 调用 (保留 in)。
时间: $\mathcal{O}(N\sqrt{Q})$

e8a06b, 62 lines

```
void add(int ind, int end) { ... } // 加入 a[ind](end 为 0 或 1)
void del(int ind, int end) { ... } // 删除 a[ind]
int calc() { ... } // 计算当前答案

vi mo(vector<pii> Q) {
    int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
    vi s((int)Q.size()), res = s;
#define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
    iota(s.begin(), s.end(), 0);
    sort(s.begin(), s.end(), [&](int s, int t){ return K(Q[s])
        < K(Q[t]); });

    for (int qi : s) {
        pii q = Q[qi];

        while (L > q.first) add(--L, 0);
        while (R < q.second) add(R++, 1);
        while (L < q.first) del(L++, 0);
        while (R > q.second) del(--R, 1);
        res[qi] = calc();
    }

    return res;
}

vi moTree(vector<array<int, 2>> Q, vector<vi> &ed, int root =
    0) {
    int N = (int)ed.size(), pos[2] = {}, blk = 350; // ~N/sqrt(
        Q)
    vi s((int)Q.size()), res = s, I(N), L(N), R(N), in(N), par(
        N);

    add(0, 0);
    in[0] = 1;
    auto dfs = [&](int x, int p, int dep, auto &f) -> void {
        par[x] = p;
        L[x] = N;
        if (dep) I[x] = N++;
        for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
        if (!dep) I[x] = N++;
        R[x] = N;
    };
    dfs(root, -1, 0, dfs);

#define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
    iota(s.begin(), s.end(), 0);
    sort(s.begin(), s.end(), [&](int s, int t){ return K(Q[s])
        < K(Q[t]); });

    for (int qi : s) {
        for (int end = 0; end < 2; end++) {
            int &a = pos[end], b = Q[qi][end], i = 0;
#define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
                  else { add(c, end); in[c] = 1; } a = c; }
            while (!(L[b] <= L[a] && R[a] <= R[b])) {
                I[i++] = b;
                b = par[b];
            }
            while (a != b) step(par[a]);
            while (i--) step(I[i]);
            if (end) res[qi] = calc();
        }
    }

    return res;
}
```

数值方法 (4)

4.1 多项式与递推

Polynomial.h

a8d1ee, 25 lines

```
struct Poly {
    vector<double> a;
```

```
    double operator()(double x) const {
        double val = 0;
        for (int i = (int)a.size(); i--;) (val *= x) += a[i];
        return val;
    }

    void diff() {
        for (int i = 1; i < (int)a.size(); i++) a[i - 1] = i *
             a[i];
        a.pop_back();
    }

    void divroot(double x0) {
        double b = a.back(), c;
        a.back() = 0;
        for (int i = (int)a.size() - 1; i--;) {
            c = a[i];
            a[i] = a[i + 1] * x0 + b;
            b = c;
        }
        a.pop_back();
    }
};
```

PolyRoots.h

描述: 求多项式实根。
用法: polyRoots({{2,-3,1}},-1e9,1e9) // 解 x^2-3x+2 = 0
时间: $\mathcal{O}\left(n^2 \log(1/\epsilon)\right)$

"Polynomial.h" 286290, 25 lines

```
vector<double> polyRoots(Poly p, double xmin, double xmax) {
    if ((int)p.a.size() == 2) return {-p.a[0] / p.a[1]};
    vector<double> ret;
    Poly der = p;
    der.diff();
    auto dr = polyRoots(der, xmin, xmax);
    dr.push_back(xmin - 1);
    dr.push_back(xmax + 1);
    sort(dr.begin(), dr.end());

    for (int i = 0; i < (int)dr.size() - 1; i++) {
        double l = dr[i], h = dr[i + 1];
        bool sign = p(l) > 0;
        if (sign ^ (p(h) > 0)) {
            for (int it = 0; it < 60; it++) { // while (h - l >
                 1e-8)
                double m = (l + h) / 2, f = p(m);
                if ((f <= 0) ^ sign) l = m;
                else h = m;
            }
            ret.push_back((l + h) / 2);
        }
    }

    return ret;
}
```

PolyInterpolate.h

描述: 给定 n 个点 (x[i], y[i]), 计算过这些点的 n-1 次多项式 p:$p(x) = a[0] * x^0 + ... + a[n-1] * x^{n-1}$。为了数值精度, 取 $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \ldots n-1$。
时间: $\mathcal{O}\left(n^2\right)$

99ae72, 22 lines

```
using vd = vector<double>;
vd interpolate(vd x, vd y, int n) {
    vd res(n), temp(n);

    for (int k = 0; k < n - 1; k++) {
        for (int i = k + 1; i < n; i++) {
            y[i] = (y[i] - y[k]) / (x[i] - x[k]);
        }
    }
    double last = 0;
    temp[0] = 1;

    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            res[i] += y[k] * temp[i];
            swap(last, temp[i]);
            temp[i] -= last * x[k];
        }
    }

    return res;
}
```

BerlekampMassey.h

描述: 从前 $2n$ 项恢复任意 n 阶线性递推关系。适合在暴力求出前几项后猜线性递推。在任何域上都应可用, 但不保证浮点的数值稳定性。输出大小 $\leq n$。
用法: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}
时间: $\mathcal{O}\left(N^2\right)$

"../number-theory/ModPow.h" ea0a0e, 26 lines

```
vector<ll> berlekampMassey(vector<ll> s) {
    int n = (int)s.size(), L = 0, m = 0;
    vector<ll> C(n), B(n), T;
    C[0] = B[0] = 1;

    ll b = 1;
    for (int i = 0; i < n; i++) {
        m++;
        ll d = s[i] % mod;
        for (int j = 1; j < L + 1; j++) d = (d + C[j] * s[i - j
             ]) % mod;
        if (!d) continue;
        T = C;
        ll coef = d * modpow(b, mod - 2) % mod;
        for (int j = m; j < n; j++) C[j] = (C[j] - coef * B[j -
             m]) % mod;
        if (2 * L > i) continue;
        L = i + 1 - L;
        B = T;
        b = d;
        m = 0;
    }

    C.resize(L + 1);
    C.erase(C.begin());
    for (ll &x : C) x = (mod - x) % mod;
    return C;
}
```

LinearRecurrence.h

描述: 计算 n 阶线性递推 $S[i] = \sum_j S[i-j-1]tr[j]$ 的第 k 项, 给定 $S[0 \ldots \geq n-1]$ 与 $tr[0 \ldots n-1]$。比矩阵乘法更快。配合 Berlekamp–Massey 使用效果很好。
用法: linearRec({0, 1}, {1, 1}, k) // 第 k 个斐波那契数
时间: $\mathcal{O}\left(n^2 \log k\right)$

44c0ed, 36 lines

```
using Poly = vector<ll>;
ll linearRec(Poly S, Poly tr, ll k) {
    int n = (int)tr.size();

    auto combine = [&](Poly a, Poly b) {
        Poly res(n * 2 + 1);

        for (int i = 0; i < n + 1; i++) {
            for (int j = 0; j < n + 1; j++) {
                res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
            }
        }
        for (int i = 2 * n; i > n; i--) {
            for (int j = 0; j < n; j++) {
                res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[
                    j]) % mod;
            }
        }
        res.resize(n + 1);
        return res;
    };

    Poly pol(n + 1), e(pol);
    pol[0] = e[1] = 1;

    for (k++; k; k /= 2) {
        if (k % 2) pol = combine(pol, e);
        e = combine(e, e);
    }

    ll res = 0;
    for (int i = 0; i < n; i++) {
        res = (res + pol[i + 1] * S[i]) % mod;
    }

    return res;
}
```

4.2 最优化

GoldenSectionSearch.h

描述: 在区间 $[a, b]$ 上求使函数 f 最小的自变量, 假设 f 在该区间上单峰, 即只有一个局部极小且无局部极大。结果的最大误差为 eps。稍作改动即可用于求最大值。离散版本见杂项章的 TernarySearch.h。
用法: double func(double x) { return 4+x+.3*x*x; }
double xmin = gss(-1000,1000,func);
时间: $\mathcal{O}\left(\log((b-a)/\epsilon)\right)$

0cd951, 23 lines

```
double gss(double a, double b, double (*f)(double)) {
    double r = (sqrt(5) - 1) / 2, eps = 1e-7;
    double x1 = b - r * (b - a), x2 = a + r * (b - a);
    double f1 = f(x1), f2 = f(x2);

    while (b - a > eps) {
        if (f1 < f2) { // 改成 > 即可求最大值
            b = x2;
            x2 = x1;
            f2 = f1;
            x1 = b - r * (b - a);
            f1 = f(x1);
        } else {
            a = x1;
            x1 = x2;
            f1 = f2;
            x2 = a + r * (b - a);
            f2 = f(x2);
        }
    }

    return a;
}
```

HillClimbing.h

描述: 单峰函数的穷人版优化。

286b70, 21 lines

```
using P = array<double, 2>;

template <class F>
pair<double, P> hillClimb(P start, F f) {
```

```
    pair<double, P> cur(f(start), start);

    for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
        for (int j = 0; j < 100; j++) {
            for (int dx = -1; dx < 2; dx++) {
                for (int dy = -1; dy < 2; dy++) {
                    P p = cur.second;
                    p[0] += dx * jmp;
                    p[1] += dy * jmp;
                    cur = min(cur, make_pair(f(p), p));
                }
            }
        }
    }

    return cur;
}
```

Integrate.h

描述: 使用 Simpson 法则在区间上对函数做简单积分。误差应与 h^4 成正比, 不过实际中你会想验证: 当 epsilon 变化时结果在所需精度内保持稳定。

c09056, 10 lines

```
template <class F>
double quad(double a, double b, F f, const int n = 1000) {
    double h = (b - a) / 2 / n, v = f(a) + f(b);

    for (int i = 1; i < n * 2; i++) {
        v += f(a + i * h) * (i & 1 ? 4 : 2);
    }

    return v * h / 3;
}
```

IntegrateAdaptive.h

描述: 使用自适应 Simpson 法则的快速积分。

用法: double sphereVolume = quad(-1, 1, [](double x) {
return quad(-1, 1, [&](double y) {
return quad(-1, 1, [&](double z) {
return x*x + y*y + z*z < 1; });});});

f57493, 17 lines

```
using d = double;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6

template <class F>
d rec(F &f, d a, d b, d eps, d S) {
    d c = (a + b) / 2;
    d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;

    if (abs(T - S) <= 15 * eps || b - a < 1e-10)
        return T + (T - S) / 15;
    return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2, S2
        );
}

template <class F>
d quad(d a, d b, F f, d eps = 1e-8) {
    return rec(f, a, b, eps, S(a, b));
}
```

Simplex.h

描述: 求解一般线性最大化问题: 最大化 $c^T x$, 约束 $Ax \leq b$,$x \geq 0$。无解返回 -inf, 有任意好的解返回 inf, 否则返回 $c^T x$ 的最大值。输入向量会被设为最优的 x(无界时, 为满足约束的任意解)。不保证数值稳定性。为获得更好性能, 定义变量时让 $x = 0$ 可行。

用法: vvd A = {{1,-1}, {-1,1}, {-1,-2}};
vd b = {1,1,-4}, c = {-1,-1}, x;
T val = LPSolver(A, b, c).solve(x);

时间: $\mathcal{O}(NM * \#pivots)$, 一次 pivot 例如一次边松弛。一般情况下为 $\mathcal{O}(2^n)$。

d8b10e, 93 lines

```
using T = double; // long double、Rational、double + mod<P>……
using vd = vector<T>;
using vvd = vector<vd>;

const T eps = 1e-8, inf = 1 / .0;
#define MP make_pair
#define ltj(X) if (s == -1 || MP(X[j], N[j]) < MP(X[s], N[s]))
    s = j

struct LPSolver {
    int m, n;
    vi N, B;
    vvd D;

    LPSolver(const vvd &A, const vd &b, const vd &c) :
        m((int)b.size()), n((int)c.size()), N(n + 1), B(m), D(m
            + 2, vd(n + 2)) {
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) D[i][j] = A[i][j];
        }
        for (int i = 0; i < m; i++) {
            B[i] = n + i;
            D[i][n] = -1;
            D[i][n + 1] = b[i];
        }
        for (int j = 0; j < n; j++) {
            N[j] = j;
            D[m][j] = -c[j];
        }
        N[n] = -1;
        D[m + 1][n] = 1;
    }

    void pivot(int r, int s) {
        T *a = D[r].data(), inv = 1 / a[s];
        for (int i = 0; i < m + 2; i++) {
            if (i != r && abs(D[i][s]) > eps) {
                T *b = D[i].data(), inv2 = b[s] * inv;
                for (int j = 0; j < n + 2; j++) b[j] -= a[j] *
                    inv2;
                b[s] = a[s] * inv2;
            }
        }
        for (int j = 0; j < n + 2; j++) {
            if (j != s) D[r][j] *= inv;
        }
        for (int i = 0; i < m + 2; i++) {
            if (i != r) D[i][s] *= -inv;
        }
        D[r][s] = inv;
        swap(B[r], N[s]);
    }

    bool simplex(int phase) {
        int x = m + phase - 1;
        for (;;) {
            int s = -1;
            for (int j = 0; j < n + 1; j++) {
                if (N[j] != -phase) ltj(D[x]);
            }
            if (D[x][s] >= -eps) return true;
            int r = -1;
            for (int i = 0; i < m; i++) {
                if (D[i][s] <= eps) continue;
                if (r == -1 || MP(D[i][n + 1] / D[i][s], B[i])
                            < MP(D[r][n + 1] / D[r][s], B[r]))
                    r = i;
            }
            if (r == -1) return false;
            pivot(r, s);
        }
    }

    T solve(vd &x) {
        int r = 0;
        for (int i = 1; i < m; i++) {
            if (D[i][n + 1] < D[r][n + 1]) r = i;
        }
        if (D[r][n + 1] < -eps) {
            pivot(r, n);
            if (!simplex(2) || D[m + 1][n + 1] < -eps) return -
                inf;
            for (int i = 0; i < m; i++) {
                if (B[i] == -1) {
                    int s = 0;
                    for (int j = 1; j < n + 1; j++) ltj(D[i]);
                    pivot(i, s);
                }
            }
        }
        bool ok = simplex(1);
        x = vd(n);
        for (int i = 0; i < m; i++) {
            if (B[i] < n) x[B[i]] = D[i][n + 1];
        }
        return ok ? D[m][n + 1] : inf;
    }
};
```

4.3 矩阵

Determinant.h

描述: 计算矩阵的行列式。会破坏矩阵。

时间: $\mathcal{O}(N^3)$

bfa65c, 25 lines

```
double det(vector<vector<double>> &a) {
    int n = (int)a.size();
    double res = 1;

    for (int i = 0; i < n; i++) {
        int b = i;
        for (int j = i + 1; j < n; j++) {
            if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
        }
        if (i != b) {
            swap(a[i], a[b]);
            res *= -1;
        }
        res *= a[i][i];
        if (res == 0) return 0;
        for (int j = i + 1; j < n; j++) {
            double v = a[j][i] / a[i][i];
            if (v != 0) {
                for (int k = i + 1; k < n; k++) a[j][k] -= v *
                    a[i][k];
            }
        }
    }

    return res;
}
```

IntDeterminant.h

描述: 使用模运算计算行列式。去掉取模即可得到纯整数版本。

时间: $\mathcal{O}(N^3)$

0f2390, 23 lines

```
const ll mod = 12345;
ll det(vector<vector<ll>> &a) {
    int n = (int)a.size();
```

```
    ll ans = 1;

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            while (a[j][i] != 0) { // gcd 步骤
                ll t = a[i][i] / a[j][i];
                if (t) {
                    for (int k = i; k < n; k++)
                        a[i][k] = (a[i][k] - a[j][k] * t) % mod
                            ;
                }
                swap(a[i], a[j]);
                ans *= -1;
            }
        }
        ans = ans * a[i][i] % mod;
        if (!ans) return 0;
    }

    return (ans + mod) % mod;
}
```

SolveLinear.h

描述: 解 $A * x = b$。若有多解, 返回任意一个。返回秩; 无解则返回 -1。A 与 b 中的数据会被破坏。
时间: $\mathcal{O}\left(n^2 m\right)$

10f7cb, 47 lines

```
using vd = vector<double>;
const double eps = 1e-12;

int solveLinear(vector<vd> &A, vd &b, vd &x) {
    int n = (int)A.size(), m = (int)x.size(), rank = 0, br, bc;
    if (n) assert((int)A[0].size() == m);
    vi col(m);
    iota(col.begin(), col.end(), 0);

    for (int i = 0; i < n; i++) {
        double v, bv = 0;
        for (int r = i; r < n; r++) {
            for (int c = i; c < m; c++) {
                if ((v = fabs(A[r][c])) > bv) {
                    br = r;
                    bc = c;
                    bv = v;
                }
            }
        }
        if (bv <= eps) {
            for (int j = i; j < n; j++) {
                if (fabs(b[j]) > eps) return -1;
            }
            break;
        }
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        for (int j = 0; j < n; j++) swap(A[j][i], A[j][bc]);
        bv = 1 / A[i][i];
        for (int j = i + 1; j < n; j++) {
            double fac = A[j][i] * bv;
            b[j] -= fac * b[i];
            for (int k = i + 1; k < m; k++) A[j][k] -= fac * A[
                i][k];
        }
        rank++;
    }

    x.assign(m, 0);
    for (int i = rank; i--;) {
        b[i] /= A[i][i];
        x[col[i]] = b[i];
        for (int j = 0; j < i; j++) b[j] -= A[j][i] * b[i];
    }
    return rank; // (rank < m 时有多解)
}
```

SolveLinear2.h

描述: 要从 SolveLinear 中恢复 x 的所有唯一确定的值, 做如下修改:

"SolveLinear.h" 98b7a6, 11 lines

```
for (int j = 0; j < n; j++) // 替代 for (int j = i + 1; j < n;
     j++)
    if (j != i)
// ... 然后末尾改为:
x.assign(m, undefined);
for (int i = 0; i < rank; i++) {
    for (int j = rank; j < m; j++) {
        if (fabs(A[i][j]) > eps) goto fail;
    }
    x[col[i]] = b[i] / A[i][i];
fail:;
}
```

SolveLinearBinary.h

描述: 在 $\mathbb{F}_2$ 上解 $Ax = b$。若有多解, 返回任意一个。返回秩; 无解返回 -1。会破坏 A 与 b。
时间: $\mathcal{O}\left(n^2 m\right)$

3a6090, 45 lines

```
using bs = bitset<1000>;

int solveLinear(vector<bs> &A, vi &b, bs &x, int m) {
    int n = (int)A.size(), rank = 0, br;
    assert(m <= (int)x.size());
    vi col(m);
    iota(col.begin(), col.end(), 0);

    for (int i = 0; i < n; i++) {
        for (br = i; br < n; br++) {
            if (A[br].any()) break;
        }
        if (br == n) {
            for (int j = i; j < n; j++) {
                if (b[j]) return -1;
            }
            break;
        }
        int bc = (int)A[br]._Find_next(i - 1);
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        for (int j = 0; j < n; j++) {
            if (A[j][i] != A[j][bc]) {
                A[j].flip(i);
                A[j].flip(bc);
            }
        }
        for (int j = i + 1; j < n; j++) {
            if (A[j][i]) {
                b[j] ^= b[i];
                A[j] ^= A[i];
            }
        }
        rank++;
    }

    x = bs();
    for (int i = rank; i--;) {
        if (!b[i]) continue;
        x[col[i]] = 1;
        for (int j = 0; j < i; j++) b[j] ^= A[j][i];
    }
    return rank; // (rank < m 时有多解)
}
```

MatrixInverse.h

描述: 求矩阵 A 的逆。返回秩; 除非奇异 (rank < n), 结果存入 A。很容易推广到素模数; 对素数幂, 反复令 $A^{-1} = A^{-1}(2I - AA^{-1})$ (mod p^k), 其中 A^{-1} 初值为 A 模 p 的逆,k 每步翻倍。
时间: $\mathcal{O}\left(n^3\right)$

014b7a, 53 lines

```
int matInv(vector<vector<double>> &A) {
    int n = (int)A.size();
    vi col(n);
    vector<vector<double>> tmp(n, vector<double>(n));

    for (int i = 0; i < n; i++) {
        tmp[i][i] = 1;
        col[i] = i;
    }

    for (int i = 0; i < n; i++) {
        int r = i, c = i;
        for (int j = i; j < n; j++) {
            for (int k = i; k < n; k++) {
                if (fabs(A[j][k]) > fabs(A[r][c])) {
                    r = j;
                    c = k;
                }
            }
        }
        if (fabs(A[r][c]) < 1e-12) return i;
        A[i].swap(A[r]);
        tmp[i].swap(tmp[r]);
        for (int j = 0; j < n; j++) {
            swap(A[j][i], A[j][c]);
            swap(tmp[j][i], tmp[j][c]);
        }
        swap(col[i], col[c]);
        double v = A[i][i];
        for (int j = i + 1; j < n; j++) {
            double f = A[j][i] / v;
            A[j][i] = 0;
            for (int k = i + 1; k < n; k++) A[j][k] -= f * A[i
                ][k];
            for (int k = 0; k < n; k++) tmp[j][k] -= f * tmp[i
                ][k];
        }
        for (int j = i + 1; j < n; j++) A[i][j] /= v;
        for (int j = 0; j < n; j++) tmp[i][j] /= v;
        A[i][i] = 1;
    }

    for (int i = n - 1; i > 0; i--) {
        for (int j = 0; j < i; j++) {
            double v = A[j][i];
            for (int k = 0; k < n; k++) tmp[j][k] -= v * tmp[i
                ][k];
        }
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) A[col[i]][col[j]] = tmp[i][
            j];
    }

    return n;
}
```

Tridiagonal.h

描述: $x = \text{tridiagonal}(d, p, q, b)$ 求解方程组

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

它可用于求解如下形式的问题

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i,\ 1 \le i \le n,$$

其中 a_0, a_{n+1}, b_i, c_i 与 d_i 已知。此时 a 可由下式得到

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, ..., -1, 1\}, \{0, c_1, c_2, \ldots, c_n\}, \{b_1, b_2, \ldots, b_n, 0\}, \{a_0, d_1, d_2, \ldots, d_n, a_{n+1}\}).$$

解不唯一时会失败。
若对所有 i 有 $|d_i| > |p_i| + |q_{i-1}|$, 或 $|d_i| > |p_{i-1}| + |q_i|$, 或矩阵正定, 则算法数值稳定, 不再需要 tr 与 diag[i] == 0 的检查。
时间: $\mathcal{O}(N)$

470b5d, 30 lines

```
using T = double;
vector<T> tridiagonal(vector<T> diag, const vector<T> &super,
        const vector<T> &sub, vector<T> b) {
    int n = (int)b.size();
    vi tr(n);

    for (int i = 0; i < n - 1; i++) {
        if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i]
            == 0
            b[i + 1] -= b[i] * diag[i + 1] / super[i];
            if (i + 2 < n) b[i + 2] -= b[i] * sub[i + 1] /
                super[i];
            diag[i + 1] = sub[i];
            tr[++i] = 1;
        } else {
            diag[i + 1] -= super[i] * sub[i] / diag[i];
            b[i + 1] -= b[i] * sub[i] / diag[i];
        }
    }
    for (int i = n; i--;) {
        if (tr[i]) {
            swap(b[i], b[i - 1]);
            diag[i - 1] = diag[i];
            b[i] /= super[i - 1];
        } else {
            b[i] /= diag[i];
            if (i) b[i - 1] -= b[i] * super[i - 1];
        }
    }

    return b;
}
```

4.4 傅里叶变换

FastFourierTransform.h

描述: fft(a) 对所有 k 计算 $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$。N 必须是 2 的幂。用于卷积: conv(a, b) = c, 其中 $c[x] = \sum a[i]b[x-i]$。复数或多个向量的卷积: 先 FFT, 逐点相乘, 除以 n, 反转 (start+1, end), 再 FFT 变换回去。若 $(\sum a_i^2 + \sum b_i^2)\log_2 N < 9 \cdot 10^{14}$, 舍入是安全的 (实际中 10^{16}; 随机输入更高)。否则, 请使用 NTT/FFTMod。
时间: $\mathcal{O}(N \log N)$,$N = |A| + |B|$($N = 2^{22}$ 时约 1s)

25fbf6, 44 lines

```
using C = complex<double>;
using vd = vector<double>;
void fft(vector<C> &a) {
    int n = (int)a.size(), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1); // (^ 用 double 快 10%)

    for (static int k = 2; k < n; k *= 2) {
        R.resize(n);
        rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);

        for (int i = k; i < 2 * k; i++) rt[i] = R[i] = i & 1 ?
            R[i / 2] * x : R[i / 2];
    }
    vi rev(n);
    for (int i = 0; i < n; i++) rev[i] = (rev[i / 2] | (i & 1)
        << L) / 2;
    for (int i = 0; i < n; i++) {
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    }
    for (int k = 1; k < n; k *= 2) {
        for (int i = 0; i < n; i += 2 * k) {
            for (int j = 0; j < k; j++) {
                C z = rt[j+k] * a[i+j+k]; // (手工展开可快 25%)
                a[i + j + k] = a[i + j] - z;
                a[i + j] += z;
            }
        }
    }
}

vd conv(const vd &a, const vd &b) {
    if (a.empty() || b.empty()) return {};
    vd res((int)a.size() + (int)b.size() - 1);
    int L = 32 - __builtin_clz((int)res.size()), n = 1 << L;
    vector<C> in(n), out(n);
    copy(a.begin(), a.end(), in.begin());
    for (int i = 0; i < (int)b.size(); i++) in[i].imag(b[i]);
    fft(in);
    for (C &x : in) x *= x;
    for (int i = 0; i < n; i++) out[i] = in[-i & (n - 1)] -
        conj(in[i]);
    fft(out);
    for (int i = 0; i < (int)res.size(); i++) res[i] = imag(out
        [i]) / (4 * n);
    return res;
}
```

FastFourierTransformMod.h

描述: 更高精度的 FFT, 可用于任意整数的卷积, 只要 $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$(实际中 10^{16} 或更高也行)。输入必须在 [0, mod) 内。
时间: $\mathcal{O}(N \log N)$, 其中 $N = |A| + |B|$(比 NTT 或 FFT 慢一倍)

"FastFourierTransform.h" 033ba6, 29 lines

```
using vl = vector<ll>;
template <int M>
vl convMod(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    vl res((int)a.size() + (int)b.size() - 1);
    int B = 32 - __builtin_clz((int)res.size()), n = 1 << B,
        cut = int(sqrt(M));
    vector<C> L(n), R(n), outs(n), outl(n);

    for (int i = 0; i < (int)a.size(); i++) L[i] = C((int)a[i]
        / cut, (int)a[i] % cut);
    for (int i = 0; i < (int)b.size(); i++) R[i] = C((int)b[i]
        / cut, (int)b[i] % cut);
    fft(L);
    fft(R);

    for (int i = 0; i < n; i++) {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
    }
    fft(outl);
    fft(outs);

    for (int i = 0; i < (int)res.size(); i++) {
        ll av = ll(real(outl[i]) + .5), cv = ll(imag(outs[i]) +
            .5);
        ll bv = ll(imag(outl[i]) + .5) + ll(real(outs[i]) + .5)
            ;
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }

    return res;
}
```

NumberTheoreticTransform.h

描述: ntt(a) 对所有 k 计算 $\hat{f}(k) = \sum_x a[x] g^{xk}$, 其中 $g = \text{root}^{(mod-1)/N}$。N 必须是 2 的幂。适用于形如 $2^a b + 1$ 的特定素数的卷积, 此时卷积结果大小至多 2^a。任意模数见 FFTMod。conv(a, b) = c, 其中 $c[x] = \sum a[i]b[x-i]$。手动卷积: 对输入做 NTT, 逐点相乘, 除以 n, 反转 (start+1, end), 再做 NTT 变回。输入必须在 [0, mod) 内。
时间: $\mathcal{O}(N \log N)$

"../number-theory/ModPow.h" cba6fb, 46 lines

```
const ll mod = (119 << 23) + 1, root = 62; // = 998244353
// 对 p < 2^30 还有例如 5 << 25、7 << 26、479 << 21
// 以及 483 << 21(同根)。后两个 > 10^9。
using vl = vector<ll>;
void ntt(vl &a) {
    int n = (int)a.size(), L = 31 - __builtin_clz(n);
    static vl rt(2, 1);

    for (static int k = 2, s = 2; k < n; k *= 2, s++) {
        rt.resize(n);
        ll z[] = {1, modpow(root, mod >> s)};

        for (int i = k; i < 2 * k; i++) rt[i] = rt[i / 2] * z[i
            & 1] % mod;
    }
    vi rev(n);
    for (int i = 0; i < n; i++) rev[i] = (rev[i / 2] | (i & 1)
        << L) / 2;
    for (int i = 0; i < n; i++) {
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    }
    for (int k = 1; k < n; k *= 2) {
        for (int i = 0; i < n; i += 2 * k) {
            for (int j = 0; j < k; j++) {
                ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[
                    i + j];
                a[i + j + k] = ai - z + (z > ai ? mod : 0);
                ai += (ai + z >= mod ? z - mod : z);
            }
        }
    }
}

vl conv(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    int s = (int)a.size() + (int)b.size() - 1, B = 32 -
        __builtin_clz(s), n = 1 << B;
    int inv = modpow(n, mod - 2);
    vl L(a), R(b), out(n);
    L.resize(n);
    R.resize(n);
    ntt(L);
    ntt(R);
```

```
    for (int i = 0; i < n; i++) {
        out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
    }
    ntt(out);
    return {out.begin(), out.begin() + s};
}
```

FastSubsetTransform.h

描述: 变换到可在如下形式下快速卷积的基: $c[z] = \sum_{z=x\oplus y} a[x] \cdot b[y]$, 其中 $\oplus$ 为 AND、OR、XOR 之一。a 的大小必须是 2 的幂。
时间: $\mathcal{O}(N \log N)$

4cb86c, 21 lines

```
void FST(vi &a, bool inv) {
    for (int n = (int)a.size(), step = 1; step < n; step *= 2)
        {
        for (int i = 0; i < n; i += 2 * step) {
            for (int j = i; j < i + step; j++) {
                int &u = a[j], &v = a[j + step];
                tie(u, v) = inv ? pii(v - u, u) : pii(v, u + v)
                    ; // AND
                inv ? pii(v, u - v) : pii(u + v, u); // OR
                pii(u + v, u - v);                   // XOR
            }
        }
    }
    if (inv) for (int& x : a) x /= sz(a); // XOR only
}

vi conv(vi a, vi b) {
    FST(a, 0);
    FST(b, 0);
    for (int i = 0; i < (int)a.size(); i++) a[i] *= b[i];
    FST(a, 1);
    return a;
}
```

数论 (5)

5.1 模运算

ModularArithmetic.h

描述: 模运算的运算符重载。需要先把 mod 设为某个数, 然后即可使用该结构体。

"euclid.h" ea1ac7, 25 lines

```
const ll mod = 17; // 改成其它数
struct Mod {
    ll x;

    Mod(ll y) : Mod(y % mod + mod, 0) {}
    Mod(ll y, int) : x(y < mod ? y : y - mod) {}

    Mod operator+(Mod b) { return {x + b.x, 0}; }
    Mod operator-(Mod b) { return {x - b.x + mod, 0}; }
    Mod operator*(Mod b) { return {x * b.x % mod, 0}; }
    Mod operator/(Mod b) { return *this * invert(b); }

    Mod invert(Mod a) {
        ll x, y, g = euclid(a.x, mod, x, y);
        assert(g == 1);
        return x;
    }

    Mod operator^(ll e) {
        if (!e) return 1;
        Mod r = *this ^ (e / 2);
        r = r * r;
        return e & 1 ? *this * r : r;
    }
};
```

ModInverse.h

描述: 模逆元的预处理。假设 LIM <= mod, 且 mod 是素数。

5a9ee9, 7 lines

```
const ll mod = 1000000007, LIM = 200000;
ll *inv = new ll[LIM] - 1;
inv[1] = 1;

for (int i = 2; i < LIM; i++) {
    inv[i] = mod - (mod / i) * inv[mod % i] % mod;
}
```

ModPow.h

描述: 快速幂取模。

aae547, 11 lines

```
const ll mod = 1000000007; // 用 const 会更快

ll modpow(ll b, ll e) {
    ll ans = 1;

    for (; e; b = b * b % mod, e /= 2) {
        if (e & 1) ans = ans * b % mod;
    }

    return ans;
}
```

ModLog.h

描述: 返回最小的 $x > 0$ 使 $a^x = b \pmod m$, 若不存在则返回 -1。modLog(a,1,m) 可用来计算 a 的阶。
时间: $\mathcal{O}\left(\sqrt{m}\right)$

237801, 16 lines

```
ll modLog(ll a, ll b, ll m) {
    ll n = (ll)sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<ll, ll> A;

    while (j <= n && (e = f = e * a % m) != b % m) {
        A[e * b % m] = j++;
    }
    if (e == b % m) return j;
    if (__gcd(m, e) == __gcd(m, b)) {
        for (int i = 2; i < n + 2; i++) {
            if (A.count(e = e * f % m)) return n * i - A[e];
        }
    }

    return -1;
}
```

ModSum.h

描述: 取模等差数列的求和。
modsum(to, c, k, m) = $\sum_{i=0}^{to-1} (ki + c)\%m$。divsum 类似, 但用于向下取整除法。
时间: $\log(m)$, 常数较大。

aba7f7, 18 lines

```
using ull = unsigned long long;

ull sumsq(ull to) { return to / 2 * ((to - 1) | 1); }

ull divsum(ull to, ull c, ull k, ull m) {
    ull res = k / m * sumsq(to) + c / m * to;
    k %= m;
    c %= m;
    if (!k) return res;
    ull to2 = (to * k + c) / m;
    return res + (to - 1) * to2 - divsum(to2, m - 1 - c, m, k);
}

ll modsum(ull to, ll c, ll k, ll m) {
    c = ((c % m) + m) % m;
    k = ((k % m) + m) % m;
    return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

描述: 对 $0 \le a, b \le c \le 7.2 \cdot 10^{18}$ 计算 $a \cdot b \bmod c$(或 $a^b \bmod c$)。
时间: modmul 为 $\mathcal{O}(1)$,modpow 为 $\mathcal{O}(\log b)$

85c1f5, 16 lines

```
using ull = unsigned long long;

ull modmul(ull a, ull b, ull M) {
    ll ret = a * b - M * ull(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (ll)M);
}

ull modpow(ull b, ull e, ull mod) {
    ull ans = 1;

    for (; e; b = modmul(b, b, mod), e /= 2) {
        if (e & 1) ans = modmul(ans, b, mod);
    }

    return ans;
}
```

ModSqrt.h

描述: 求模平方根的 Tonelli-Shanks 算法。求 x 使 $x^2 = a \pmod p$($-x$ 是另一解)。
时间: 最坏 $\mathcal{O}\left(\log^2 p\right)$, 对多数 p 为 $\mathcal{O}(\log p)$

"ModPow.h" 6f8e6b, 34 lines

```
ll sqrt(ll a, ll p) {
    a %= p;
    if (a < 0) a += p;
    if (a == 0) return 0;
    assert(modpow(a, (p - 1) / 2, p) == 1); // 否则无解
    if (p % 4 == 3) return modpow(a, (p + 1) / 4, p);

    // 若 p % 8 == 5,a^(n+3)/8 或 2^(n+3)/8 * 2^(n-1)/4 也可行
    ll s = p - 1, n = 2;
    int r = 0, m;

    while (s % 2 == 0) {
        r++;
        s /= 2;
    }
    while (modpow(n, (p - 1) / 2, p) != p - 1) {
        n++;
    }

    ll x = modpow(a, (s + 1) / 2, p);
    ll b = modpow(a, s, p), g = modpow(n, s, p);

    for (;; r = m) {
        ll t = b;
        for (m = 0; m < r && t != 1; m++) {
            t = t * t % p;
        }
        if (m == 0) return x;
        ll gs = modpow(g, 1LL << (r - m - 1), p);
        g = gs * gs % p;
        x = x * gs % p;
        b = b * g % p;
    }
}
```

5.2 素性

FastEratosthenes.h

描述: 生成小于 LIM 的所有素数的筛法。
时间: LIM=1e9 约 1.5s

7d48c1, 30 lines

```
const int LIM = 1e6;
bitset<LIM> isPrime;

vi eratosthenes() {
    const int S = (int)round(sqrt(LIM)), R = LIM / 2;
    vi pr = {2}, sieve(S + 1);
    pr.reserve(int(LIM / log(LIM) * 1.1));
    vector<pii> cp;

    for (int i = 3; i <= S; i += 2) {
        if (!sieve[i]) {
            cp.push_back({i, i * i / 2});
            for (int j = i * i; j <= S; j += 2 * i) sieve[j] =
                1;
        }
    }

    for (int L = 1; L <= R; L += S) {
        array<bool, S> block{};
        for (auto &[p, idx] : cp) {
            for (int i = idx; i < S + L; idx = (i += p)) block[
                i - L] = 1;
        }

        for (int i = 0; i < min(S, R - L); i++) {
            if (!block[i]) pr.push_back((L + i) * 2 + 1);
        }
    }

    for (int i : pr) isPrime[i] = 1;
    return pr;
}
```

MillerRabin.h

描述: 确定性 Miller-Rabin 素性测试。保证对 $7 \cdot 10^{18}$ 以内的数正确; 更大的数请用 Python 并随机扩展 A。
时间: 约 7 倍于 $a^b \mod c$ 的复杂度。

"ModMulLL.h" 3ba57b, 15 lines

```
bool isPrime(ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022
        },
        s = __builtin_ctzll(n - 1), d = n >> s;

    for (ull a : A) { // ^ 计算末尾零的个数
        ull p = modpow(a % n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--) {
            p = modmul(p, p, n);
        }
        if (p != n - 1 && i != s) return 0;
    }

    return 1;
}
```

Factor.h

描述: Pollard-rho 随机化因数分解算法。返回一个数的质因子, 顺序任意 (如 2299 -> {11, 19, 11})。
时间: $\mathcal{O}\left(n^{1/4}\right)$, 含小因子的数更快。

"ModMulLL.h", "MillerRabin.h" e85d2b, 26 lines

```
ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return modmul(x, x, n) + i; };

    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) {
            x = ++i;
            y = f(x);
        }
        if ((q = modmul(prd, max(x, y) - min(x, y), n))) prd =
            q;
        x = f(x);
        y = f(f(y));
    }

    return __gcd(prd, n);
}

vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x);
    auto r = factor(n / x);
    l.insert(l.end(), r.begin(), r.end());
    return l;
}
```

5.3 整除

euclid.h

描述: 求整数 x 与 y, 使 $ax+by = \gcd(a,b)$。若只需要 gcd, 请改用内置的 __gcd。若 a 与 b 互素, 则 x 是 $a \pmod b$ 的逆元。

33ba8f, 5 lines

```
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a / b * x, d;
}
```

CRT.h

描述: 中国剩余定理。
crt(a, m, b, n) 计算满足 $x \equiv a \pmod m$、$x \equiv b \pmod n$ 的 x。若 $|a| < m$ 且 $|b| < n$, 则所得 x 满足 $0 \le x < \mathrm{lcm}(m,n)$。假设 $mn < 2^{62}$。
时间: $\log(n)$

"euclid.h" a48462, 10 lines

```
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) {
        swap(a, b);
        swap(m, n);
    }
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // 否则无解
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m * n / g : x;
}
```

5.3.1 Bézout 等式

对 $a \neq 0$, $b \neq 0$, 则 $d = gcd(a,b)$ 是使下式存在整数解的最小正整数

$$ax + by = d$$

若 (x,y) 是一组解, 则全部解为

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

描述: 欧拉 ϕ 函数定义为 $\phi(n) :=$ 不超过 n 且与 n 互素的正整数个数。$\phi(1) = 1$,p 为素数 $\Rightarrow \phi(p^k) = (p-1)p^{k-1}$,$m,n$ 互素 $\Rightarrow \phi(mn) = \phi(m)\phi(n)$。若 $n = p_1^{k_1} p_2^{k_2} ... p_r^{k_r}$, 则 $\phi(n) = (p_1-1)p_1^{k_1-1} ... (p_r-1)p_r^{k_r-1}$。$\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$。
$\sum_{d|n} \phi(d) = n$,$\sum_{1 \le k \le n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$
欧拉定理:a,n 互素 $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$。
费马小定理:p 为素数 $\Rightarrow a^{p-1} \equiv 1 \pmod p$ $\forall a$。

d9b072, 14 lines

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
    for (int i = 0; i < LIM; i++) {
        phi[i] = i & 1 ? i : i / 2;
    }

    for (int i = 3; i < LIM; i += 2) {
        if (phi[i] == i) {
            for (int j = i; j < LIM; j += i) phi[j] -= phi[j] /
                i;
        }
    }
}
```

5.4 分数

ContinuedFractions.h

描述: 给定 N 与非负实数 $x \ge 0$, 求满足 $p,q \le N$ 的最佳有理近似 p/q。它将满足 $|p/q - x| \le 1/qN$。
对相邻的收敛项,$p_{k+1}q_k - q_{k+1}p_k = (-1)^k$。($p_k/q_k$ 在 $> x$ 与 $< x$ 之间交替。)
若 x 是有理数,y 最终会变为 ∞; 若 x 是二次多项式的根, 则各 a 最终会呈循环。
时间: $\mathcal{O}(\log N)$

59018b, 26 lines

```
using d = double; // N ~ 1e7 时用 double;N ~ 1e9 时用 long
    double
pair<ll, ll> approximate(d x, ll N) {
    ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX;
    d y = x;

    for (;;) {
        ll lim = min(P ? (N - LP) / P : inf, Q ? (N - LQ) / Q :
            inf),
          a = (ll)floor(y), b = min(a, lim),
          NP = b * P + LP, NQ = b * Q + LQ;

        if (a > b) {
            // 若 b > a/2,存在给出更好近似的半收敛项;
            // 若 b = a/2,则可能有一个。
            // 这里返回 {P, Q} 可获得更规范的近似。
            return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)
                Q)) ?
                make_pair(NP, NQ) : make_pair(P, Q);
        }
        if (abs(y = 1 / (y - (d)a)) > 3 * N) {
            return {NP, NQ};
        }
        LP = P;
        P = NP;
        LQ = Q;
        Q = NQ;
    }
}
```

FracBinarySearch.h

描述: 给定 f 与 N, 求满足 $f(p/q)$ 为真且 $p,q \le N$ 的最小分数 $p/q \in [0,1]$。若 f 找到精确解时可以抛异常, 那样就可以去掉 N。
用法: fracBS([](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}

时间: $\mathcal{O}(\log(N))$ 27ab3e, 29 lines

```
struct Frac { ll p, q; };

template <class F>
Frac fracBS(F f, ll N) {
    bool dir = 1, A = 1, B = 1;
    Frac lo{0, 1}, hi{1, 1}; // 把 hi 设为 1/0 可搜索 (0, N]
    if (f(lo)) return lo;
    assert(f(hi));

    while (A || B) {
        ll adv = 0, step = 1; // dir 为真时移动 hi,否则移动 lo
        for (int si = 0; step; (step *= 2) >>= si) {
            adv += step;
            Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
            if (abs(mid.p) > N || mid.q > N || dir == !f(mid))
                {
                adv -= step;
                si = 2;
            }
        }
        hi.p += lo.p * adv;
        hi.q += lo.q * adv;
        dir = !dir;
        swap(lo, hi);
        A = B;
        B = !!adv;
    }

    return dir ? hi : lo;
}
```

5.5 毕达哥拉斯三元组

毕达哥拉斯三元组可由下式唯一生成

$$a = k \cdot (m^2 - n^2),\ b = k \cdot (2mn),\ c = k \cdot (m^2 + n^2),$$

其中 $m > n > 0$,$k > 0$,$m \perp n$, 且 m、n 一奇一偶。

5.6 素数

$p = 962592769$ 满足 $2^{21} \mid p - 1$, 可能有用。哈希用 970592641(31 位数)、31443539979727(45 位)、3006703054056749 (52 位)。小于 1 000 000 的素数有 78498 个。

原根对任何素数幂 p^a 都存在, 除了 $p = 2, a > 2$; 共有 $\phi(\phi(p^a))$ 个。对 $p = 2, a > 2$, 群 $\mathbb{Z}_{2^a}^{\times}$ 同构于 $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$。

5.7 估计

$\sum_{d|n} d = O(n \log\log n)$。

n 的约数个数:$n < 5e4$ 时至多约 100 个,$n < 1e7$ 时 500 个,$n < 1e10$ 时 2000 个,$n < 1e19$ 时 200 000 个。

5.8 莫比乌斯函数

$$\mu(n) = \begin{cases} 0 & n \text{ 不是无平方因子数} \\ 1 & n \text{ 有偶数个质因子} \\ -1 & n \text{ 有奇数个质因子} \end{cases}$$

莫比乌斯反演:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

其它有用的形式:

$\sum_{d|n} \mu(d) = [n = 1]$(非常有用)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$

$g(n) = \sum_{1 \le m \le n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \le m \le n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$

组合数学 (6)

6.1 排列

6.1.1 阶乘

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800

n	11	12	13	14	15	16	17
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14

n	20	25	30	40	50	100	150	171
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX

IntPerm.h

描述: 排列 $\to$ 整数。(不保序。) 整数 $\to$ 排列可以用查表。

时间: $\mathcal{O}(n)$ ed2504, 10 lines

```
int permToInt(vi &v) {
    int use = 0, i = 0, r = 0;

    for (int x : v) {
        r = r * ++i + __builtin_popcount(use & -(1 << x));
        use |= 1 << x; // (注意:是负号,不是取反!)
    }

    return r;
}
```

6.1.2 轮换

设 $g_S(n)$ 为轮换长度全部属于集合 S 的 n-排列个数。则

$$\sum_{n=0}^{\infty} g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{n \in S} \frac{x^n}{n}\right)$$

6.1.3 错排

集合的排列中, 没有任何元素出现在原位置的排列。

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

6.1.4 Burnside 引理

给定对称群 G 与集合 X,X 的元素在对称意义下的个数等于

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

其中 X^g 是被 g 固定的元素 ($g.x = x$)。

若 $f(n)$ 计数长度为 n 的某种" 构型", 用 $G = \mathbb{Z}_n$ 消去旋转对称后得到

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n} \sum_{k|n} f(k) \phi(n/k).$$

6.2 划分与子集

6.2.1 划分数

把 n 写成正整数之和的方法数, 不计顺序。

$$p(0) = 1,\ p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	∼2e5	∼2e8

6.2.2 Lucas 定理

设 n, m 为非负整数,p 为素数。写 $n = n_k p^k + ... + n_1 p + n_0$、$m = m_k p^k + ... + m_1 p + m_0$。则 $\binom{n}{m} \equiv \prod_{i=0}^{k} \binom{n_i}{m_i} \pmod{p}$。

6.2.3 二项式系数

multinomial.h

描述: 计算 $\binom{k_1 + \cdots + k_n}{k_1, k_2, \ldots, k_n} = \frac{(\sum k_i)!}{k_1! k_2! \cdots k_n!}$。 14ca53, 11 lines

```
ll multinomial(vi &v) {
    ll c = 1, m = v.empty() ? 1 : v[0];

    for (int i = 1; i < (int)v.size(); i++) {
        for (int j = 0; j < v[i]; j++) {
            c = c * ++m / (j + 1);
        }
    }

    return c;
}
```

6.3 常用数列

6.3.1 伯努利数

伯努利数的 EGF 为 $B(t) = \frac{t}{e^t - 1}$(可用 FFT 计算)。

$B[0, \ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

幂和:

$$\sum_{i=1}^{n} n^m = \frac{1}{m+1} \sum_{k=0}^{m} \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

无穷和的 Euler-Maclaurin 公式:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty} \frac{B_k}{k!} f^{(k-1)}(m)$$

$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

6.3.2 第一类斯特林数

n 个元素恰好构成 k 个轮换的排列数。

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$
$$\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\dots(x+n-1)$$

$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$

6.3.3 Eulerian 数

恰有 k 个元素大于前一个元素的排列 $\pi \in S_n$ 的个数。k 个 j 使 $\pi(j) > \pi(j+1)$,$k+1$ 个 j 使 $\pi(j) \geq j$,k 个 j 使 $\pi(j) > j$。

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$
$$E(n,0) = E(n,n-1) = 1$$
$$E(n,k) = \sum_{j=0}^{k}(-1)^j \binom{n+1}{j}(k+1-j)^n$$

6.3.4 第二类斯特林数

把 n 个不同元素划分为恰好 k 组的方法数。

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$
$$S(n,1) = S(n,n) = 1$$
$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

6.3.5 Bell 数

n 个不同元素的划分数总和。$B(n) =$
$1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$。对素数 p,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod{p}$$

6.3.6 带标号的无根树

n 个顶点: n^{n-2}
k 棵已有树 (大小 n_i): $n_1 n_2 \cdots n_k n^{k-2}$
度数为 d_i: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

6.3.7 Catalan 数

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$
$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$

- $n \times n$ 网格中不过对角线的单调路径。
- n 对括号的正确嵌套串。
- 有 $n+1$ 个叶子 (0 或 2 个儿子) 的二叉树。
- 有 $n+1$ 个顶点的有序树。
- 把 $n+2$ 边形用不相交对角线剖分成三角形的方法数。
- $[n]$ 的无三项递增子序列的排列数。

图论 (7)

7.1 基础

BellmanFord.h

描述: 计算可能含负权边的图中从 s 出发的最短路。不可达点的距离为 inf; 能通过负权环到达的点的距离为 -inf。假设 $V^2 \max|w_i| < \sim 2^{63}$。
时间: $\mathcal{O}(VE)$
403e37, 30 lines

```
const ll inf = LLONG_MAX;
struct Ed {
    int a, b, w;
    int s() { return a < b ? a : -a; }
};
struct Node { ll dist = inf; int prev = -1; };

void bellmanFord(vector<Node> &nodes, vector<Ed> &eds, int s) {
    nodes[s].dist = 0;
    sort(eds.begin(), eds.end(), [](Ed a, Ed b) { return a.s()
        < b.s(); });

    int lim = (int)nodes.size() / 2 + 2; // 顶点打乱后为 /3+100
    for (int i = 0; i < lim; i++) {
        for (Ed ed : eds) {
            Node cur = nodes[ed.a], &dest = nodes[ed.b];
            if (abs(cur.dist) == inf) continue;
            ll d = cur.dist + ed.w;
            if (d < dest.dist) {
                dest.prev = ed.a;
                dest.dist = (i < lim - 1 ? d : -inf);
            }
        }
    }
    for (int i = 0; i < lim; i++) {
        for (Ed e : eds) {
            if (nodes[e.a].dist == -inf)
                nodes[e.b].dist = -inf;
        }
    }
}
```

FloydWarshall.h

描述: 计算可能含负权边的有向图的全源最短路。输入为距离矩阵 m, 其中 i 与 j 不相邻时 $m[i][j] =$ inf。输出时 $m[i][j]$ 被设为 i 到 j 的最短距离: 无路径为 inf, 路径经过负权环则为 -inf。
时间: $\mathcal{O}(N^3)$
b5ff69, 25 lines

```
const ll inf = 1LL << 62;
void floydWarshall(vector<vector<ll>> &m) {
    int n = (int)m.size();
    for (int i = 0; i < n; i++) m[i][i] = min(m[i][i], 0LL);

    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (m[i][k] != inf && m[k][j] != inf) {
                    auto newDist = max(m[i][k] + m[k][j], -inf)
                        ;
                    m[i][j] = min(m[i][j], newDist);
                }
            }
        }
    }
    for (int k = 0; k < n; k++) {
        if (m[k][k] < 0) {
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    if (m[i][k] != inf && m[k][j] != inf) m[i][
                        j] = -inf;
                }
            }
        }
    }
}
```

TopoSort.h

描述: 拓扑排序。输入为有向图。输出为顶点的排列, 使所有边都从左指向右。若有环, 返回列表的大小将小于 n ——从环可达的节点不会返回。
时间: $\mathcal{O}(|V| + |E|)$
81867e, 17 lines

```
vi topoSort(const vector<vi> &gr) {
    vi indeg((int)gr.size()), q;

    for (auto &li : gr) {
        for (int x : li) indeg[x]++;
    }
    for (int i = 0; i < (int)gr.size(); i++) {
        if (indeg[i] == 0) q.push_back(i);
    }
    for (int j = 0; j < (int)q.size(); j++) {
        for (int x : gr[q[j]]) {
            if (--indeg[x] == 0) q.push_back(x);
        }
    }

    return q;
}
```

7.2 网络流

PushRelabel.h

描述: 使用最高标号选择与 gap 启发式的 push-relabel 算法。实际中相当快。要得到实际的流量, 只看正值即可。
时间: $\mathcal{O}\left(V^2\sqrt{E}\right)$
a447c2, 74 lines

```
struct PushRelabel {
    struct Edge {
        int dest, back;
        ll f, c;
    };

    vector<vector<Edge>> g;
    vector<ll> ec;
    vector<Edge *> cur;
    vector<vi> hs;
    vi H;

    PushRelabel(int n) : g(n), ec(n), cur(n), hs(2 * n), H(n) {
        }

    void addEdge(int s, int t, ll cap, ll rcap = 0) {
        if (s == t) return;
        g[s].push_back({t, (int)g[t].size(), 0, cap});
        g[t].push_back({s, (int)g[s].size() - 1, 0, rcap});
    }

    void addFlow(Edge &e, ll f) {
        Edge &back = g[e.dest][e.back];
        if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
        e.f += f;
        e.c -= f;
        ec[e.dest] += f;
        back.f -= f;
        back.c += f;
```

```
        ec[back.dest] -= f;
    }

    ll calc(int s, int t) {
        int v = (int)g.size();
        H[s] = v;
        ec[t] = 1;
        vi co(2 * v);
        co[0] = v - 1;
        for (int i = 0; i < v; i++) cur[i] = g[i].data();
        for (Edge &e : g[s]) addFlow(e, e.c);

        for (int hi = 0;;) {
            while (hs[hi].empty()) {
                if (!hi--) return -ec[s];
            }
            int u = hs[hi].back();
            hs[hi].pop_back();
            while (ec[u] > 0) { // 排出 u 的盈余
                if (cur[u] == g[u].data() + (int)g[u].size()) {
                    H[u] = 1e9;
                    for (Edge &e : g[u]) {
                        if (e.c && H[u] > H[e.dest] + 1) {
                            H[u] = H[e.dest] + 1;
                            cur[u] = &e;
                        }
                    }
                    if (++co[H[u]], !--co[hi] && hi < v) {
                        for (int i = 0; i < v; i++) {
                            if (hi < H[i] && H[i] < v) {
                                --co[H[i]];
                                H[i] = v + 1;
                            }
                        }
                    }
                    hi = H[u];
                }
                else if (cur[u]->c && H[u] == H[cur[u]->dest] +
                      1)
                    addFlow(*cur[u], min(ec[u], cur[u]->c));
                else ++cur[u];
            }
        }
    }

    bool leftOfMinCut(int a) { return H[a] >= (int)g.size(); }
};
```

MinCostMaxFlow.h

描述: 最小费用最大流。若费用可能为负, 在 maxflow 前调用 setpi, 但注意不支持负费用环。要得到实际的流量, 只看正值即可。
时间: $\mathcal{O}(FE\log(V))$,F 为最大流;setpi 为 $\mathcal{O}(VE)$。

9d7ec2, 100 lines

```
#include <bits/extc++.h>

const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };

    int N;
    vector<vector<edge>> ed;
    vi seen;
    vector<ll> dist, pi;
    vector<edge *> par;

    MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N)
        {}

    void addEdge(int from, int to, ll cap, ll cost) {
        if (from == to) return;
        ed[from].push_back(edge{from, to, (int)ed[to].size(),
            cap, cost, 0});
        ed[to].push_back(edge{to, from, (int)ed[from].size() -
            1, 0, -cost, 0});
    }

    void path(int s) {
        fill(seen.begin(), seen.end(), 0);
        fill(dist.begin(), dist.end(), INF);
        dist[s] = 0;
        ll di;

        __gnu_pbds::priority_queue<pair<ll, int>> q;
        vector<decltype(q)::point_iterator> its(N);
        q.push({0, s});

        while (!q.empty()) {
            s = q.top().second;
            q.pop();
            seen[s] = 1;
            di = dist[s] + pi[s];
            for (edge &e : ed[s]) {
                if (!seen[e.to]) {
                    ll val = di - pi[e.to] + e.cost;
                    if (e.cap - e.flow > 0 && val < dist[e.to])
                        {
                        dist[e.to] = val;
                        par[e.to] = &e;
                        if (its[e.to] == q.end())
                            its[e.to] = q.push({-dist[e.to], e.
                                to});
                        else
                            q.modify(its[e.to], {-dist[e.to], e
                                .to});
                    }
                }
            }
        }
        for (int i = 0; i < N; i++) pi[i] = min(pi[i] + dist[i
            ], INF);
    }

    pair<ll, ll> maxflow(int s, int t) {
        ll totflow = 0, totcost = 0;

        while (path(s), seen[t]) {
            ll fl = INF;
            for (edge *x = par[t]; x; x = par[x->from])
                fl = min(fl, x->cap - x->flow);

            totflow += fl;
            for (edge *x = par[t]; x; x = par[x->from]) {
                x->flow += fl;
                ed[x->to][x->rev].flow -= fl;
            }
        }
        for (int i = 0; i < N; i++) {
            for (edge &e : ed[i]) totcost += e.cost * e.flow;
        }
        return {totflow, totcost / 2};
    }

    // 若某些费用可能为负,在 maxflow 前调用:
    void setpi(int s) { // (否则可以删掉这段)
        fill(pi.begin(), pi.end(), INF);
        pi[s] = 0;
        int it = N, ch = 1;
        ll v;

        while (ch-- && it--) {
            for (int i = 0; i < N; i++) {
                if (pi[i] != INF) {
                    for (edge &e : ed[i]) {
                        if (e.cap) {
                            if ((v = pi[i] + e.cost) < pi[e.to
                                ]) {
                                pi[e.to] = v;
                                ch = 1;
                            }
                        }
                    }
                }
            }
        }
        assert(it >= 0); // 负费用环
    }
};
```

EdmondsKarp.h

描述: 复杂度保证为 $O(VE^2)$ 的网络流算法。要得到每条边的流量, 比较前后容量, 只取正值即可。

611c4e, 36 lines

```
template <class T>
T edmondsKarp(vector<unordered_map<int, T>> &graph, int source,
      int sink) {
    assert(source != sink);
    T flow = 0;
    vi par((int)graph.size()), q = par;

    for (;;) {
        fill(par.begin(), par.end(), -1);
        par[source] = 0;
        int ptr = 1;
        q[0] = source;

        for (int i = 0; i < ptr; i++) {
            int x = q[i];
            for (auto e : graph[x]) {
                if (par[e.first] == -1 && e.second > 0) {
                    par[e.first] = x;
                    q[ptr++] = e.first;
                    if (e.first == sink) goto out;
                }
            }
        }
        return flow;
out:
        T inc = numeric_limits<T>::max();
        for (int y = sink; y != source; y = par[y])
            inc = min(inc, graph[par[y]][y]);

        flow += inc;
        for (int y = sink; y != source; y = par[y]) {
            int p = par[y];
            if ((graph[p][y] -= inc) <= 0) graph[p].erase(y);
            graph[y][p] += inc;
        }
    }
}
```

MinCut.h

描述: 运行最大流后, 从 s 到 t 的最小割的左侧即为从 s 出发、只走残余容量为正的边所能到达的顶点集合。

GlobalMinCut.h

描述: 求无向图的全局最小割 (以邻接矩阵表示)。
时间: $\mathcal{O}\left(V^3\right)$

cd4787, 24 lines

```
pair<int, vi> globalMinCut(vector<vi> mat) {
    pair<int, vi> best = {INT_MAX, {}};
    int n = (int)mat.size();
    vector<vi> co(n);
    for (int i = 0; i < n; i++) co[i] = {i};

    for (int ph = 1; ph < n; ph++) {
        vi w = mat[0];
        size_t s = 0, t = 0;
        for (int it = 0; it < n - ph; it++) { // 用优先队列可从
            O(V^2) 优化到 O(E log V)
            w[t] = INT_MIN;
            s = t;
            t = max_element(w.begin(), w.end()) - w.begin();
            for (int i = 0; i < n; i++) w[i] += mat[t][i];
        }
        best = min(best, {w[t] - mat[t][t], co[t]});
        co[s].insert(co[s].end(), co[t].begin(), co[t].end());
        for (int i = 0; i < n; i++) mat[s][i] += mat[t][i];
        for (int i = 0; i < n; i++) mat[i][s] = mat[s][i];
        mat[0][t] = INT_MIN;
    }

    return best;
}
```

GomoryHu.h

描述: 给定表示无向流网络的边表, 返回 Gomory-Hu 树的边。任意两点间的最大流等于 Gomory-Hu 树路径上的最小边权。
时间: $\mathcal{O}(V)$ 次流计算

"PushRelabel.h" 1232b2, 16 lines

```
using Edge = array<ll, 3>;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
    vector<Edge> tree;
    vi par(N);

    for (int i = 1; i < N; i++) {
        PushRelabel D(N); // Dinic 也可以
        for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
        tree.push_back({i, par[i], D.calc(i, par[i])});
        for (int j = i + 1; j < N; j++) {
            if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] =
                i;
        }
    }

    return tree;
}
```

7.3 匹配

HopcroftKarp.h

描述: 快速的二分图匹配算法。图 g 应为左侧部分的邻居表, r 应为大小与右侧部分相同、初值全为 -1 的 vector。返回匹配的大小。$r[i]$ 为右侧顶点 i 的匹配对象, 未匹配时为 -1。
时间: $\mathcal{O}\left(E\sqrt{V}\right)$

caf326, 35 lines

```
int hopcroftKarp(vector<vi> &g, vi &r) {
    int n = (int)g.size(), res = 0;
    vi l(n, -1), q(n), d(n);

    auto dfs = [&](auto f, int u) -> bool {
        int t = exchange(d[u], 0) + 1;
        for (int v : g[u]) {
            if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
                return l[u] = v, r[v] = u, 1;
        }
        return 0;
    };

    for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
        for (int i = 0; i < n; i++) {
            if (l[i] == -1) {
                q[t++] = i;
                d[i] = 1;
            }
        }
        for (int i = 0; i < t; i++) {
            for (int v : g[q[i]]) {
                if (r[v] == -1) f = 1;
                else if (!d[r[v]]) {
                    d[r[v]] = d[q[i]] + 1;
                    q[t++] = r[v];
                }
            }
        }
        if (!f) return res;
        for (int i = 0; i < n; i++) {
            if (l[i] == -1) res += dfs(dfs, i);
        }
    }
}
```

DFSMatching.h

描述: 简单的二分图匹配算法。图 g 应为左侧部分的邻居表,$btoa$ 应为大小与右侧部分相同、初值全为 -1 的 vector。返回匹配的大小。$btoa[i]$ 为右侧顶点 i 的匹配对象, 未匹配时为 -1。
用法: vi btoa(m, -1); dfsMatching(g, btoa);
时间: $\mathcal{O}(VE)$

627f76, 26 lines

```
bool find(int j, vector<vi> &g, vi &btoa, vi &vis) {
    if (btoa[j] == -1) return 1;
    vis[j] = 1;
    int di = btoa[j];
    for (int e : g[di]) {
        if (!vis[e] && find(e, g, btoa, vis)) {
            btoa[e] = di;
            return 1;
        }
    }
    return 0;
}
int dfsMatching(vector<vi> &g, vi &btoa) {
    vi vis;

    for (int i = 0; i < (int)g.size(); i++) {
        vis.assign((int)btoa.size(), 0);
        for (int j : g[i]) {
            if (find(j, g, btoa, vis)) {
                btoa[j] = i;
                break;
            }
        }
    }
    return (int)btoa.size() - (int)count(btoa.begin(), btoa.end
        (), -1);
}
```

MinimumVertexCover.h

描述: 求二分图的最小点覆盖。大小与最大匹配相同, 其补集为最大独立集。

"DFSMatching.h" 3eb8e4, 32 lines

```
vi cover(vector<vi> &g, int n, int m) {
    vi match(m, -1);
    int res = dfsMatching(g, match);
    vector<bool> lfound(n, true), seen(m);

    for (int it : match) {
        if (it != -1) lfound[it] = false;
    }
    vi q, cover;
    for (int i = 0; i < n; i++) {
        if (lfound[i]) q.push_back(i);
    }
    while (!q.empty()) {
        int i = q.back();
        q.pop_back();
        lfound[i] = 1;
        for (int e : g[i]) {
            if (!seen[e] && match[e] != -1) {
                seen[e] = true;
                q.push_back(match[e]);
            }
        }
    }
    for (int i = 0; i < n; i++) {
        if (!lfound[i]) cover.push_back(i);
    }
    for (int i = 0; i < m; i++) {
        if (seen[i]) cover.push_back(n + i);
    }
    assert((int)cover.size() == res);
    return cover;
}
```

WeightedMatching.h

描述: 给定带权二分图, 把左侧每个节点与右侧某节点匹配, 使没有节点出现在两个匹配中且边权之和最小。输入 cost[N][M], 其中 cost[i][j] = L[i] 与 R[j] 匹配的代价, 返回 (最小代价, match), 其中 L[i] 与 R[match[i]] 匹配。代价取反可用于求最大权。要求 $N \leq M$。
时间: $\mathcal{O}\left(N^2M\right)$

fb6d16, 38 lines

```
template <class T>
pair<T, vi> weightedMatching(vector<vector<T>> &C) {
    int i = (int)C.size(), m = i ? (int)C[0].size() : 0, c, s,
        r;
    vector<T> dist(m), pot(m);
    vi match(i), rev(m, -1), cols(m), prev(m);
    T d = 0, nd, cost = 0;

    while (i--) {
        for (int c = 0; c < m; c++) {
            dist[c] = C[i][c];
            cols[c] = c;
            prev[c] = i;
        }
        for (s = 0;;) {
            for (int j = s; j < m; j++) {
                c = cols[j];
                nd = dist[c] - pot[c];
                if (j == s || d > nd) {
                    d = nd;
                    swap(cols[s], cols[j]);
                }
            }
            if ((r = rev[c = cols[s++]]) == -1) break;
            for (int j = 0; j < m; j++) {
```

```
                if (dist[j] > (nd = C[r][j] - C[r][c] + dist[c
                    ])) {
                    dist[j] = nd;
                    prev[j] = r;
                }
            }
        }
        cost += dist[c];
        while (s--) pot[cols[s]] = dist[cols[s]] - d;
        for (; r != i; swap(c, match[r]))
            r = rev[c] = prev[c];
    }

    return {cost, match};
}
```

GeneralMatching.h

描述: 一般图的匹配。以 N/mod 的概率失败。
时间: $\mathcal{O}(N^3)$

"../numerical/MatrixInverse-mod.h" 865fc3, 62 lines

```
vector<pii> generalMatching(int N, vector<pii> &ed) {
    vector<vector<ll>> mat(N, vector<ll>(N)), A;

    for (pii pa : ed) {
        int a = pa.first, b = pa.second, r = rand() % mod;
        mat[a][b] = r;
        mat[b][a] = (mod - r) % mod;
    }

    int r = matInv(A = mat), M = 2 * N - r, fi, fj;
    assert(r % 2 == 0);

    if (M != N) {
        do {
            mat.resize(M, vector<ll>(M));
            for (int i = 0; i < N; i++) {
                mat[i].resize(M);
                for (int j = N; j < M; j++) {
                    int r = rand() % mod;
                    mat[i][j] = r;
                    mat[j][i] = (mod - r) % mod;
                }
            }
        } while (matInv(A = mat) != M);
    }

    vi has(M, 1);
    vector<pii> ret;

    for (int it = 0; it < M / 2; it++) {
        for (int i = 0; i < M; i++) {
            if (has[i]) {
                for (int j = i + 1; j < M; j++) {
                    if (A[i][j] && mat[i][j]) {
                        fi = i;
                        fj = j;
                        goto done;
                    }
                }
            }
        }
        assert(0);
done:
        if (fj < N) ret.emplace_back(fi, fj);
        has[fi] = has[fj] = 0;

        for (int sw = 0; sw < 2; sw++) {
            ll a = modpow(A[fi][fj], mod - 2);
            for (int i = 0; i < M; i++) {
                if (has[i] && A[i][fj]) {
                    ll b = A[i][fj] * a % mod;
                    for (int j = 0; j < M; j++) {
                        A[i][j] = (A[i][j] - A[fi][j] * b) %
                            mod;
                    }
                }
            }
            swap(fi, fj);
        }
    }

    return ret;
}
```

7.4 DFS 算法

SCC.h

描述: 求有向图的强连通分量。若顶点 u, v 属于同一分量, 则从 v 可达 u, 反之亦然。
用法: scc(graph, [&](vi& v) { ... }) 按逆拓扑序
访问所有分量。comp[i] 保存节点 i 的分量编号
(一个分量只会连向编号更小的分量)。ncomps 为分量个数。
时间: $\mathcal{O}(E+V)$

8596f0, 36 lines

```
vi val, comp, z, cont;
int Time, ncomps;

template <class G, class F>
int dfs(int j, G &g, F &f) {
    int low = val[j] = ++Time, x;
    z.push_back(j);
    for (auto e : g[j]) {
        if (comp[e] < 0)
            low = min(low, val[e] ? val[e] : dfs(e, g, f));
    }

    if (low == val[j]) {
        do {
            x = z.back();
            z.pop_back();
            comp[x] = ncomps;
            cont.push_back(x);
        } while (x != j);
        f(cont);
        cont.clear();
        ncomps++;
    }
    return val[j] = low;
}

template <class G, class F>
void scc(G &g, F f) {
    int n = (int)g.size();
    val.assign(n, 0);
    comp.assign(n, -1);
    Time = ncomps = 0;
    for (int i = 0; i < n; i++) {
        if (comp[i] < 0) dfs(i, g, f);
    }
}
```

BiconnectedComponents.h

描述: 求无向图的所有点双连通分量, 并对每个分量中的边运行一次回调。在一个点双连通分量中, 任意两节点之间至少有两条内部不相交的路径 (即存在经过它们的环)。注意一个节点可以出现在多个分量中。不属于任何分量的边是桥, 即不在任何环上。

用法: int eid = 0; ed.resize(N);
for each edge (a,b) {
ed[a].emplace_back(b, eid);
ed[b].emplace_back(a, eid++); }
bicomps([&](const vi& edgelist) {...});
时间: $\mathcal{O}(E+V)$

cfc795, 37 lines

```
vi num, st;
vector<vector<pii>> ed;
int Time;

template <class F>
int dfs(int at, int par, F &f) {
    int me = num[at] = ++Time, top = me;
    for (auto [y, e] : ed[at]) {
        if (e != par) {
            if (num[y]) {
                top = min(top, num[y]);
                if (num[y] < me)
                    st.push_back(e);
            } else {
                int si = (int)st.size();
                int up = dfs(y, e, f);
                top = min(top, up);
                if (up == me) {
                    st.push_back(e);
                    f(vi(st.begin() + si, st.end()));
                    st.resize(si);
                }
                else if (up < me) st.push_back(e);
                else { /* e 是桥 */ }
            }
        }
    }
    return top;
}

template <class F>
void bicomps(F f) {
    num.assign((int)ed.size(), 0);
    for (int i = 0; i < (int)ed.size(); i++) {
        if (!num[i]) dfs(i, -1, f);
    }
}
```

2sat.h

描述: 为 2-SAT 问题的布尔变量 a, b, c,... 计算一个合法赋值, 使形如 $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&\ldots$ 的表达式为真, 或报告其不可满足。取反的变量用按位取反表示 (~x)。
用法: TwoSat ts(number of boolean variables);
ts.either(0, ~3); // 变量 0 为真或变量 3 为假
ts.setValue(2); // 变量 2 为真
ts.atMostOne({0,~1,2}); // 0、~1、2 中至多一个为真
ts.solve(); // 可满足时返回 true
ts.values[0..N-1] 保存各变量的赋值
时间: $\mathcal{O}(N+E)$, 其中 N 为布尔变量数,E 为子句数。

035108, 71 lines

```
struct TwoSat {
    int N;
    vector<vi> gr;
    vi values; // 0 = false, 1 = true

    TwoSat(int n = 0) : N(n), gr(2 * n) {}

    int addVar() { // (可选)
        gr.emplace_back();
        gr.emplace_back();
        return N++;
    }
```

```
    void either(int f, int j) {
        f = max(2 * f, -1 - 2 * f);
        j = max(2 * j, -1 - 2 * j);
        gr[f].push_back(j ^ 1);
        gr[j].push_back(f ^ 1);
    }

    void setValue(int x) { either(x, x); }

    void atMostOne(const vi &li) { // (可选)
        if ((int)li.size() <= 1) return;
        int cur = ~li[0];

        for (int i = 2; i < (int)li.size(); i++) {
            int next = addVar();
            either(cur, ~li[i]);
            either(cur, next);
            either(~li[i], next);
            cur = ~next;
        }
        either(cur, ~li[1]);
    }

    vi val, comp, z;
    int time = 0;

    int dfs(int i) {
        int low = val[i] = ++time, x;
        z.push_back(i);
        for (int e : gr[i]) {
            if (!comp[e])
                low = min(low, val[e] ? val[e] : dfs(e));
        }
        if (low == val[i]) {
            do {
                x = z.back();
                z.pop_back();
                comp[x] = low;
                if (values[x >> 1] == -1)
                    values[x >> 1] = x & 1;
            } while (x != i);
        }
        return val[i] = low;
    }

    bool solve() {
        values.assign(N, -1);
        val.assign(2 * N, 0);
        comp = val;
        for (int i = 0; i < 2 * N; i++) {
            if (!comp[i]) dfs(i);
        }
        for (int i = 0; i < N; i++) {
            if (comp[2 * i] == comp[2 * i + 1]) return 0;
        }
        return 1;
    }
};
```

EulerWalk.h

描述: 无向/有向图的欧拉路/回路算法。输入应为 (终点, 全局边编号) 的 vector, 其中无向图的正反两条边共用同一编号。返回欧拉路/回路的节点列表, 起点与终点均为 src; 若不存在回路/路, 返回空列表。要取回边编号, 给 s 和 ret 加上.second。
时间: $\mathcal{O}(V+E)$

f2edf7, 25 lines

```
vi eulerWalk(vector<vector<pii>> &gr, int nedges, int src = 0)
    {
    int n = (int)gr.size();
    vi D(n), its(n), eu(nedges), ret, s = {src};
    D[src]++; // 以允许欧拉路(而不只是回路)

    while (!s.empty()) {
        int x = s.back(), y, e, &it = its[x], end = (int)gr[x].
            size();
        if (it == end) {
            ret.push_back(x);
            s.pop_back();
            continue;
        }
        tie(y, e) = gr[x][it++];
        if (!eu[e]) {
            D[x]--;
            D[y]++;
            eu[e] = 1;
            s.push_back(y);
        }
    }
    for (int x : D) {
        if (x < 0 || (int)ret.size() != nedges + 1) return {};
    }
    return {ret.rbegin(), ret.rend()};
}
```

7.5 染色

EdgeColoring.h

描述: 给定最大度为 D 的简单无向图, 计算边的 $(D+1)$ 染色, 使相邻边不同色。(D 染色是 NP 难的, 但对二分图可以在最大度节点上反复匹配做到。)
时间: $\mathcal{O}(NM)$

3d689d, 42 lines

```
vi edgeColoring(int N, vector<pii> eds) {
    vi cc(N + 1), ret((int)eds.size()), fan(N), free(N), loc;

    for (pii e : eds) {
        cc[e.first]++;
        cc[e.second]++;
    }
    int u, v, ncols = *max_element(cc.begin(), cc.end()) + 1;
    vector<vi> adj(N, vi(ncols, -1));

    for (pii e : eds) {
        tie(u, v) = e;
        fan[0] = v;
        loc.assign(ncols, 0);
        int at = u, end = u, d, c = free[u], ind = 0, i = 0;
        while (d = free[v], !loc[d] && (v = adj[u][d]) != -1) {
            loc[d] = ++ind;
            cc[ind] = d;
            fan[ind] = v;
        }
        cc[loc[d]] = c;
        for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd
            ]) {
            swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
        }
        while (adj[fan[i]][d] != -1) {
            int left = fan[i], right = fan[++i], e = cc[i];
            adj[u][e] = left;
            adj[left][e] = u;
            adj[right][e] = -1;
            free[right] = e;
        }
        adj[u][d] = fan[i];
        adj[fan[i]][d] = u;
        for (int y : {fan[0], u, end}) {
            for (int &z = free[y] = 0; adj[y][z] != -1; z++);
        }
    }
    for (int i = 0; i < (int)eds.size(); i++) {
        for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i
            ];
    }
    return ret;
}
```

7.6 启发式

MaximalCliques.h

描述: 对图中的所有极大团运行一次回调 (输入为对称 bitset 矩阵; 不允许自环)。回调收到表示该极大团的 bitset。
时间: $\mathcal{O}\left(3^{n/3}\right)$, 稀疏图快得多

50cd1a, 19 lines

```
using B = bitset<128>;
template <class F>
void cliques(vector<B> &eds, F f, B P = ~B(), B X = {}, B R = {
    }) {
    if (!P.any()) {
        if (!X.any()) f(R);
        return;
    }
    auto q = (P | X)._Find_first();
    auto cands = P & ~eds[q];

    for (int i = 0; i < (int)eds.size(); i++) {
        if (cands[i]) {
            R[i] = 1;
            cliques(eds, f, P & eds[i], X & eds[i], R);
            R[i] = P[i] = 0;
            X[i] = 1;
        }
    }
}
```

MaximumClique.h

描述: 快速求图的最大团 (输入为对称 bitset 矩阵; 不允许自环)。可通过在补图上求团来得到最大独立集。
时间: n=155 的最坏情形随机图 (p=.90) 约 1s。稀疏图更快。

64c3c3, 74 lines

```
using vb = vector<bitset<200>>;
struct Maxclique {
    double limit = 0.025, pk = 0;
    struct Vertex { int i, d = 0; };
    using vv = vector<Vertex>;
    vb e;
    vv V;
    vector<vi> C;
    vi qmax, q, S, old;

    void init(vv &r) {
        for (auto &v : r) v.d = 0;
        for (auto &v : r) {
            for (auto j : r) v.d += e[v.i][j.i];
        }
        sort(r.begin(), r.end(), [](auto a, auto b) { return a.
            d > b.d; });
        int mxD = r[0].d;
        for (int i = 0; i < (int)r.size(); i++) r[i].d = min(i,
            mxD) + 1;
    }

    void expand(vv &R, int lev = 1) {
        S[lev] += S[lev - 1] - old[lev];
        old[lev] = S[lev - 1];

        while ((int)R.size()) {
            if ((int)q.size() + R.back().d <= (int)qmax.size())
                return;
```

```
            q.push_back(R.back().i);
            vv T;

            for (auto v : R) {
                if (e[R.back().i][v.i]) T.push_back({v.i});
            }
            if ((int)T.size()) {
                if (S[lev]++ / ++pk < limit) init(T);
                int j = 0, mxk = 1, mnk = max((int)qmax.size()
                     - (int)q.size() + 1, 1);
                C[1].clear();
                C[2].clear();

                for (auto v : T) {
                    int k = 1;
                    auto f = [&](int i) { return e[v.i][i]; };
                    while (any_of(C[k].begin(), C[k].end(), f))
                          k++;
                    if (k > mxk) {
                        mxk = k;
                        C[mxk + 1].clear();
                    }
                    if (k < mnk) T[j++].i = v.i;
                    C[k].push_back(v.i);
                }
                if (j > 0) T[j - 1].d = 0;
                for (int k = mnk; k < mxk + 1; k++) {
                    for (int i : C[k]) {
                        T[j].i = i;
                        T[j++].d = k;
                    }
                }
                expand(T, lev + 1);
            }
            else if ((int)q.size() > (int)qmax.size()) qmax = q
                 ;
            q.pop_back();
            R.pop_back();
        }
    }

    vi maxClique() {
        init(V);
        expand(V);
        return qmax;
    }

    Maxclique(vb conn) : e(conn), C((int)e.size() + 1), S((int)
         C.size()), old(S) {
        for (int i = 0; i < (int)e.size(); i++) V.push_back({i}
             );
    }
};
```

MaximumIndependentSet.h

描述: 要求图的最大独立集, 可在补图上求最大团。若图是二分图, 见 MinimumVertexCover。

7.7 树

BinaryLifting.h

描述: 计算树上的倍增跳表, 支持快速向上跳与 LCA。假设根节点指向自身。

时间: 建表 $\mathcal{O}(N \log N)$, 查询 $\mathcal{O}(\log N)$

b7ceec, 35 lines

```
vector<vi> treeJump(vi &P) {
    int on = 1, d = 1;
    while (on < (int)P.size()) {
        on *= 2;
        d++;
    }
    vector<vi> jmp(d, P);
    for (int i = 1; i < d; i++) {
        for (int j = 0; j < (int)P.size(); j++) {
            jmp[i][j] = jmp[i - 1][jmp[i - 1][j]];
        }
    }
    return jmp;
}

int jmp(vector<vi> &tbl, int nod, int steps) {
    for (int i = 0; i < (int)tbl.size(); i++) {
        if (steps & (1 << i)) nod = tbl[i][nod];
    }
    return nod;
}

int lca(vector<vi> &tbl, vi &depth, int a, int b) {
    if (depth[a] < depth[b]) swap(a, b);
    a = jmp(tbl, a, depth[a] - depth[b]);
    if (a == b) return a;
    for (int i = (int)tbl.size(); i--;) {
        int c = tbl[i][a], d = tbl[i][b];
        if (c != d) {
            a = c;
            b = d;
        }
    }
    return tbl[0][a];
}
```

LCA.h

描述: 计算树上最近公共祖先的数据结构 (以 0 为根)。C 为树的邻接表, 可以有向或无向。

时间: $\mathcal{O}(N \log N + Q)$

"../data-structures/RMQ.h" efeb77, 25 lines

```
struct LCA {
    int T = 0;
    vi time, path, ret;
    RMQ<int> rmq;

    LCA(vector<vi> &C) : time((int)C.size()), rmq((dfs(C, 0,
         -1), ret)) {}

    void dfs(vector<vi> &C, int v, int par) {
        time[v] = T++;
        for (int y : C[v]) {
            if (y != par) {
                path.push_back(v);
                ret.push_back(time[v]);
                dfs(C, y, v);
            }
        }
    }

    int lca(int a, int b) {
        if (a == b) return a;
        tie(a, b) = minmax(time[a], time[b]);
        return path[rmq.query(a, b)];
    }
    //dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];
         }
};
```

CompressTree.h

描述: 给定一棵有根树与节点子集 S, 通过加入至多 $|S| - 1$ 个两两 LCA 并压缩边, 求出包含全部节点的最小子树。返回 (par, orig_index) 的列表, 表示一棵以 0 为根的树。根指向自身。

时间: $\mathcal{O}(|S| \log |S|)$

"LCA.h" 267722, 25 lines

```
using vpi = vector<pair<int, int>>;
vpi compressTree(LCA &lca, const vi &subset) {
    static vi rev;
    rev.resize((int)lca.time.size());
    vi li = subset, &T = lca.time;
    auto cmp = [&](int a, int b) { return T[a] < T[b]; };
    sort(li.begin(), li.end(), cmp);
    int m = (int)li.size() - 1;

    for (int i = 0; i < m; i++) {
        int a = li[i], b = li[i + 1];
        li.push_back(lca.lca(a, b));
    }
    sort(li.begin(), li.end(), cmp);
    li.erase(unique(li.begin(), li.end()), li.end());
    for (int i = 0; i < (int)li.size(); i++) rev[li[i]] = i;
    vpi ret = {pii(0, li[0])};

    for (int i = 0; i < (int)li.size() - 1; i++) {
        int a = li[i], b = li[i + 1];
        ret.emplace_back(rev[lca.lca(a, b)], b);
    }

    return ret;
}
```

HLD.h

描述: 把树分解为顶点不相交的重链与轻边, 使任意叶子到根的路径上至多 $\log(n)$ 条轻边。代码做的是区间加与区间最大值查询, 但可以支持路径与子树上可交换的线段树修改/查询。输入为完整的邻接表。VALS_EDGES 为真表示值存在边上而非节点上。所有值初始化为线段树默认值。根必须为 0。

时间: $\mathcal{O}\left((\log N)^2\right)$

"../data-structures/LazySegmentTree.h" 8d7953, 58 lines

```
template <bool VALS_EDGES>
struct HLD {
    int N, tim = 0;
    vector<vi> adj;
    vi par, siz, rt, pos;
    Node *tree;

    HLD(vector<vi> adj_)
        : N((int)adj_.size()), adj(adj_), par(N, -1), siz(N, 1)
            ,
          rt(N), pos(N), tree(new Node(0, N)) {
        dfsSz(0);
        dfsHld(0);
    }

    void dfsSz(int v) {
        for (int &u : adj[v]) {
            adj[u].erase(find(adj[u].begin(), adj[u].end(), v))
                ;
            par[u] = v;
            dfsSz(u);
            siz[v] += siz[u];
            if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
        }
    }

    void dfsHld(int v) {
        pos[v] = tim++;
        for (int u : adj[v]) {
            rt[u] = (u == adj[v][0] ? rt[v] : u);
```

```
            dfsHld(u);
        }
    }

    template <class B>
    void process(int u, int v, B op) {
        for (;; v = par[rt[v]]) {
            if (pos[u] > pos[v]) swap(u, v);
            if (rt[u] == rt[v]) break;
            op(pos[rt[v]], pos[v] + 1);
        }
        op(pos[u] + VALS_EDGES, pos[v] + 1);
    }

    void modifyPath(int u, int v, int val) {
        process(u, v, [&](int l, int r) { tree->add(l, r, val);
            });
    }

    int queryPath(int u, int v) { // 依题目修改
        int res = -1e9;
        process(u, v, [&](int l, int r) {
            res = max(res, tree->query(l, r));
        });
        return res;
    }

    int querySubtree(int v) { // modifySubtree 类似
        return tree->query(pos[v] + VALS_EDGES, pos[v] + siz[v
            ]);
    }
};
```

LinkCutTree.h

描述: 维护一片无根树 (森林)。可以加边、删边 (只要结果仍是森林), 并查询两个节点是否在同一棵树中。
时间: 所有操作均摊 $\mathcal{O}(\log N)$。

f1a079, 108 lines

```
struct Node { // Splay 树。根的 pp 指向其代表树中的父节点。
    Node *p = 0, *pp = 0, *c[2];
    bool flip = 0;

    Node() {
        c[0] = c[1] = 0;
        fix();
    }
    void fix() {
        if (c[0]) c[0]->p = this;
        if (c[1]) c[1]->p = this;
        // (如需维护子树和信息等,在此更新)
    }
    void pushFlip() {
        if (!flip) return;
        flip = 0;
        swap(c[0], c[1]);
        if (c[0]) c[0]->flip ^= 1;
        if (c[1]) c[1]->flip ^= 1;
    }
    int up() { return p ? p->c[1] == this : -1; }

    void rot(int i, int b) {
        int h = i ^ b;
        Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y :
            x;
        if ((y->p = p)) p->c[up()] = y;
        c[i] = z->c[i ^ 1];
        if (b < 2) {
            x->c[h] = y->c[h ^ 1];
            y->c[h ^ 1] = x;
        }
        z->c[i ^ 1] = this;
        fix();
        x->fix();
        y->fix();
        if (p) p->fix();
        swap(pp, y->pp);
    }

    void splay() {
        for (pushFlip(); p;) {
            if (p->p) p->p->pushFlip();
            p->pushFlip();
            pushFlip();
            int c1 = up(), c2 = p->up();
            if (c2 == -1) p->rot(c1, 2);
            else p->p->rot(c2, c1 != c2);
        }
    }

    Node *first() {
        pushFlip();
        return c[0] ? c[0]->first() : (splay(), this);
    }
};

struct LinkCut {
    vector<Node> node;

    LinkCut(int N) : node(N) {}

    void link(int u, int v) { // 添加边 (u, v)
        assert(!connected(u, v));
        makeRoot(&node[u]);
        node[u].pp = &node[v];
    }
    void cut(int u, int v) { // 删除边 (u, v)
        Node *x = &node[u], *top = &node[v];
        makeRoot(top);
        x->splay();
        assert(top == (x->pp ? x->pp : x->c[0]));
        if (x->pp) x->pp = 0;
        else {
            x->c[0] = top->p = 0;
            x->fix();
        }
    }
    bool connected(int u, int v) { // u、v 是否在同一棵树中?
        Node *nu = access(&node[u])->first();
        return nu == access(&node[v])->first();
    }
    void makeRoot(Node *u) {
        access(u);
        u->splay();
        if (u->c[0]) {
            u->c[0]->p = 0;
            u->c[0]->flip ^= 1;
            u->c[0]->pp = u;
            u->c[0] = 0;
            u->fix();
        }
    }
    Node *access(Node *u) {
        u->splay();
        while (Node *pp = u->pp) {
            pp->splay();
            u->pp = 0;
            if (pp->c[1]) {
                pp->c[1]->p = 0;
                pp->c[1]->pp = pp;
            }
            pp->c[1] = u;
            pp->fix();
            u = pp;
        }
        return u;
    }
};
```

DirectedMST.h

描述: 求有向图以给定节点为根的最小生成树 (树形图)。若不存在 MST, 返回 -1。
时间: $\mathcal{O}(E \log V)$

"../data-structures/UnionFindRollback.h" cdfbfb, 76 lines

```
struct Edge { int a, b; ll w; };
struct Node {
    Edge key;
    Node *l, *r;
    ll delta;

    void prop() {
        key.w += delta;
        if (l) l->delta += delta;
        if (r) r->delta += delta;
        delta = 0;
    }
    Edge top() {
        prop();
        return key;
    }
};
Node *merge(Node *a, Node *b) {
    if (!a || !b) return a ? a : b;
    a->prop();
    b->prop();
    if (a->key.w > b->key.w) swap(a, b);
    swap(a->l, (a->r = merge(b, a->r)));
    return a;
}
void pop(Node *&a) {
    a->prop();
    a = merge(a->l, a->r);
}

pair<ll, vi> dmst(int n, int r, vector<Edge> &g) {
    RollbackUF uf(n);
    vector<Node *> heap(n);

    for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
    ll res = 0;
    vi seen(n, -1), path(n), par(n);
    seen[r] = r;
    vector<Edge> Q(n), in(n, {-1, -1}), comp;
    deque<tuple<int, int, vector<Edge>>> cycs;

    for (int s = 0; s < n; s++) {
        int u = s, qi = 0, w;
        while (seen[u] < 0) {
            if (!heap[u]) return {-1, {}};
            Edge e = heap[u]->top();
            heap[u]->delta -= e.w;
            pop(heap[u]);
            Q[qi] = e;
            path[qi++] = u;
            seen[u] = s;
            res += e.w;
            u = uf.find(e.a);
            if (seen[u] == s) {
                Node *cyc = 0;
```

```
                int end = qi, time = uf.time();
                do cyc = merge(cyc, heap[w = path[--qi]]);
                while (uf.join(u, w));
                u = uf.find(u);
                heap[u] = cyc;
                seen[u] = -1;
                cycs.push_front({u, time, {&Q[qi], &Q[end]}});
            }
        }
        for (int i = 0; i < qi; i++) in[uf.find(Q[i].b)] = Q[i
            ];
    }

    for (auto &[u, t, comp] : cycs) { // 还原解(可选)
        uf.rollback(t);
        Edge inEdge = in[u];
        for (auto &e : comp) in[uf.find(e.b)] = e;
        in[uf.find(inEdge.b)] = inEdge;
    }
    for (int i = 0; i < n; i++) par[i] = in[i].a;
    return {res, par};
}
```

7.8 数学

7.8.1 生成树个数

构造 $N \times N$ 矩阵 mat, 对每条边 $a \to b \in G$, 做 mat[a][b]–, mat[b][b]++(G 为无向图时再做 mat[b][a]–, mat[a][a]++)。删去第 i 行与第 i 列并求行列式, 即得根在 i 的有向生成树个数 (若 G 无向, 删任意一行与任意一列)。

7.8.2 Erdős–Gallai 定理

存在节点度数 $d_1 \geq \cdots \geq d_n$ 的简单图, 当且仅当 $d_1 + \cdots + d_n$ 为偶数且对每个 $k = 1 \ldots n$,

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

计算几何 (8)

8.1 几何基础

Point.h

描述: 平面上的点类。T 可取 double 或 long long。(避免用 int。) 9c840e, 37 lines

```
template <class T>
int sgn(T x) { return (x > 0) - (x < 0); }

template <class T>
struct Point {
    using P = Point;
    T x, y;

    explicit Point(T x = 0, T y = 0) : x(x), y(y) {}

    bool operator<(P p) const { return tie(x, y) < tie(p.x, p.y
        ); }
    bool operator==(P p) const { return tie(x, y) == tie(p.x, p
        .y); }
    P operator+(P p) const { return P(x + p.x, y + p.y); }
    P operator-(P p) const { return P(x - p.x, y - p.y); }
    P operator*(T d) const { return P(x * d, y * d); }
    P operator/(T d) const { return P(x / d, y / d); }
    T dot(P p) const { return x * p.x + y * p.y; }
    T cross(P p) const { return x * p.y - y * p.x; }
    T cross(P a, P b) const { return (a - *this).cross(b -
        this); }
    T dist2() const { return x * x + y * y; }
    double dist() const { return sqrt((double)dist2()); }
    // 与 x 轴的夹角,范围 [-pi, pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this / dist(); } // 使 dist()=1
    P perp() const { return P(-y, x); } // 逆时针旋转 90 度
    P normal() const { return perp().unit(); }

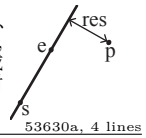
    // 返回绕原点逆时针旋转 a 弧度后的点
    P rotate(double a) const {
        return P(x * cos(a) - y * sin(a), x * sin(a) + y * cos(
            a));
    }

    friend ostream &operator<<(ostream &os, P p) {
        return os << "(" << p.x << "," << p.y << ")";
    }
};
```

lineDistance.h

描述:

返回点 p 到过点 a、b 的直线的带符号距离。从 a 指向 b 看, 左侧为正、右侧为负。a==b 时返回 nan。P 应为 Point<T> 或 Point3D<T>, 其中 T 如 double 或 long long。中间步骤使用乘积, 若用 int 或 long long 请注意溢出。对 Point3D 结果恒非负; 对 Point3D, 请对叉积结果调用.dist。



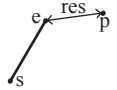
"Point.h" 53630a, 4 lines

```
template <class P>
double lineDist(P a, P b, P p) {
    return (double)(b - a).cross(p - a) / (b - a).dist();
}
```

SegmentDistance.h

描述:

返回点 p 到线段 s-e 的最短距离。



用法: Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;

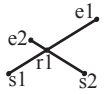
"Point.h" 4c7da9, 6 lines

```
using P = Point<double>;
double segDist(P s, P e, P p) {
    if (s == e) return (p - s).dist();
    auto d = (e - s).dist2(), t = min(d, max(.0, (p - s).dot(e
        - s)));
    return ((p - s) * d - (e - s) * t).dist() / d;
}
```

SegmentIntersection.h

描述:

若线段 s1-e1 与 s2-e2 有唯一交点, 则将其返回。若无交点, 返回空 vector。若有无穷多交点, 返回含 2 个元素的 vector, 即公共线段的两个端点。若 P 为 Point<ll> 且交点坐标不是整数, 返回的位置是错误的。中间步骤使用三个坐标的乘积, 若用 int 或 long long 请注意溢出。



用法: vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;

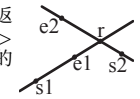
"Point.h", "OnSegment.h" f6be16, 14 lines

```
template <class P>
vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b),
         oc = a.cross(b, c), od = a.cross(b, d);
    // 检查交点是否为单个非端点
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
    if (onSegment(a, b, d)) s.insert(d);
    return {s.begin(), s.end()};
}
```

lineIntersection.h

描述:

若过 s1,e1 与 s2,e2 的直线有唯一交点, 返回 {1, 交点}。若无交点返回 {0, (0,0)}; 若有无穷多交点返回 {-1, (0,0)}。若 P 为 Point<ll> 且交点坐标不是整数, 返回的位置是错误的。中间步骤使用三个坐标的乘积, 若用 int 或 ll 请注意溢出。



用法: auto res = lineInter(s1,e1,s2,e2);
if (res.first == 1)
cout << "intersection point at " << res.second << endl;

"Point.h" a01f81, 8 lines

```
template <class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // 若平行
        return {-(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
```

sideOf.h

描述: 返回从 s 指向 e 看时 p 的位置。1/0/-1 分别对应左侧/在线上/右侧。若给出可选参数 eps, 则当 p 距直线不超过 eps 时返回 0。P 应为 Point<T>, 其中 T 如 double 或 long long。中间步骤使用乘积, 若用 int 或 long long 请注意溢出。

用法: bool left = sideOf(p1,p2,q)==1;

"Point.h" ca4665, 9 lines

```
template <class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

template <class P>
int sideOf(P s, P e, P p, double eps) {
    auto a = (e - s).cross(p - s);
    double l = (e - s).dist() * eps;
    return (a > l) - (a < -l);
}
```

OnSegment.h

描述: 返回 p 是否在线段 s-e 上 (为真当且仅当)。使用 Point<double> 时请改用 (segDist(s,e,p)<=epsilon)。

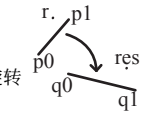
"Point.h" c597e8, 4 lines

```
template <class P>
bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

linearTransformation.h

描述:

对点 r 应用把直线 p0-p1 映到直线 q0-q1 的线性变换 (平移、旋转与缩放)。



"Point.h" d4d48b, 5 lines

```
using P = Point<double>;
P linearTransformation(P p0, P p1, P q0, P q1, P r) {
    P dp = p1 - p0, dq = q1 - q0, num(dp.cross(dq), dp.dot(dq))
        ;
```

```
    return q0 + P((r - p0).cross(num), (r - p0).dot(num)) / dp.
        dist2();
}
```

Angle.h

描述: 用于给角度排序的类 (以整点表示, 并记录绕原点旋转的圈数)。适合旋转扫描。有时也用来表示点或向量。
用法: vector<Angle> v = {w[0], w[0].t360() ...}; // 已排序
int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }
// 扫动 j, 使 (j-i) 表示以 0 和 i 为顶点的正向三角形个数

0f0602, 40 lines

```
struct Angle {
    int x, y;
    int t;

    Angle(int x, int y, int t = 0) : x(x), y(y), t(t) {}
    Angle operator-(Angle b) const { return {x - b.x, y - b.y,
        t}; }
    int half() const {
        assert(x || y);
        return y < 0 || (y == 0 && x < 0);
    }
    Angle t90() const { return {-y, x, t + (half() && x >= 0)};
        }
    Angle t180() const { return {-x, -y, t + half()}; }
    Angle t360() const { return {x, y, t + 1}; }
};

bool operator<(Angle a, Angle b) {
    // 加上 a.dist2() 与 b.dist2() 也可比较距离
    return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
           make_tuple(b.t, b.half(), a.x * (ll)b.y);
}

// 给定两个点,计算它们之间的最小角,
// 即覆盖所定义线段的角度。
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
    if (b < a) swap(a, b);
    return (b < a.t180() ?
            make_pair(a, b) : make_pair(b, a.t360()));
}

Angle operator+(Angle a, Angle b) { // 点 a + 向量 b
    Angle r(a.x + b.x, a.y + b.y, a.t);
    if (a.t180() < r) r.t--;
    return r.t180() < a ? r.t360() : r;
}

Angle angleDiff(Angle a, Angle b) { // 角度 b - 角度 a
    int tu = b.t - a.t;
    a.t = b.t;
    return {a.x * b.x + a.y * b.y, a.x * b.y - a.y * b.x, tu -
        (b < a)};
}
```

8.2 圆

CircleIntersection.h

描述: 计算两圆的交点 (至多两个)。无交点时返回 false。

"Point.h" 93169e, 14 lines

```
using P = Point<double>;
bool circleInter(P a, P b, double r1, double r2, pair<P, P> *
    out) {
    if (a == b) {
        assert(r1 != r2);
        return false;
    }
    P vec = b - a;
    double d2 = vec.dist2(), sum = r1 + r2, dif = r1 - r2,
           p = (d2 + r1 * r1 - r2 * r2) / (d2 * 2), h2 = r1 *
               r1 - p * p * d2;
    if (sum * sum < d2 || dif * dif > d2) return false;
    P mid = a + vec * p, per = vec.perp() * sqrt(fmax(0, h2) /
        d2);
    *out = {mid + per, mid - per};
    return true;
}
```

CircleTangents.h

描述: 求两圆的外公切线; 若 r2 取负则求内公切线。可返回 0、1 或 2 条切线——若一圆包含另一圆则为 0 (内公切线情形下两圆相交或相同也为 0); 若两圆相切则为 1(此时.first = .second, 切线与连心线垂直)。.first 与.second 分别为圆 1 与圆 2 上的切点。求圆与点集的切线时, 把 r2 设为 0。

"Point.h" b0153d, 14 lines

```
template <class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
    P d = c2 - c1;
    double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0) return {};
    vector<pair<P, P>> out;

    for (double sign : {-1, 1}) {
        P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
        out.push_back({c1 + v * r1, c2 + v * r2});
    }
    if (h2 == 0) out.pop_back();
    return out;
}
```

CirclePolygonIntersection.h

描述: 返回圆与逆时针多边形的交的面积。
时间: $\mathcal{O}(n)$

"../../content/geometry/Point.h" 1d75ae, 22 lines

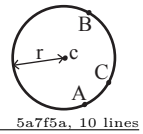
```
using P = Point<double>;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
    auto tri = [&](P p, P q) {
        auto r2 = r * r / 2;
        P d = q - p;
        auto a = d.dot(p) / d.dist2(), b = (p.dist2() - r * r)
            / d.dist2();
        auto det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        auto s = max(0., -a - sqrt(det)), t = min(1., -a + sqrt
            (det));
        if (t < 0 || 1 <= s) return arg(p, q) * r2;
        P u = p + d * s, v = q + d * (t - 1);
        return arg(p, u) * r2 + u.cross(v) / 2 + arg(v, q) * r2
            ;
    };
    auto sum = 0.0;

    for (int i = 0; i < (int)ps.size(); i++) {
        sum += tri(ps[i] - c, ps[(i + 1) % (int)ps.size()] - c)
            ;
    }

    return sum;
}
```

circumcircle.h

描述:



三角形的外接圆是过其三个顶点的圆。ccRadius 返回过点 A、B、C 的圆的半径, ccCenter 返回同一个圆的圆心。

"Point.h" 5a7f5a, 10 lines

```
using P = Point<double>;
double ccRadius(P A, P B, P C) {
    return (B - A).dist() * (C - B).dist() * (A - C).dist() /
           abs((B - A).cross(C - A)) / 2;
}

P ccCenter(P A, P B, P C) {
    P b = C - A, c = B - A;
    return A + (b * c.dist2() - c * b.dist2()).perp() / b.cross
        (c) / 2;
}
```

MinimumEnclosingCircle.h

描述: 计算包含点集的最小圆。
时间: 期望 $\mathcal{O}(n)$

"circumcircle.h" ce1731, 28 lines

```
pair<P, double> mec(vector<P> ps) {
    shuffle(ps.begin(), ps.end(), mt19937(time(0)));
    P o = ps[0];
    double r = 0, EPS = 1 + 1e-8;

    for (int i = 0; i < (int)ps.size(); i++) {
        if ((o - ps[i]).dist() > r * EPS) {
            o = ps[i];
            r = 0;

            for (int j = 0; j < i; j++) {
                if ((o - ps[j]).dist() > r * EPS) {
                    o = (ps[i] + ps[j]) / 2;
                    r = (o - ps[i]).dist();

                    for (int k = 0; k < j; k++) {
                        if ((o - ps[k]).dist() > r * EPS) {
                            o = ccCenter(ps[i], ps[j], ps[k]);
                            r = (o - ps[i]).dist();
                        }
                    }
                }
            }
        }
    }

    return {o, r};
}
```

8.3 多边形

InsidePolygon.h

描述: 返回 p 是否在多边形内。若 strict 为真, 边界上的点返回 false。中间步骤使用乘积, 请注意溢出。
用法: vector<P> v = {P{4,4}, P{1,2}, P{2,1}};
bool in = inPolygon(v, P{3, 3}, false);
时间: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h" a575c0, 13 lines

```
template <class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
    int cnt = 0, n = (int)p.size();

    for (int i = 0; i < n; i++) {
        P q = p[(i + 1) % n];
        if (onSegment(p[i], q, a)) return !strict;
        // 或者:if (segDist(p[i], q, a) <= eps) return !strict;
```

```
        cnt ^= ((a.y < p[i].y) - (a.y < q.y)) * a.cross(p[i], q
            ) > 0;
    }

    return cnt;
}
```

PolygonArea.h

描述: 返回多边形带符号面积的两倍。顺时针枚举得到负面积。T 用 int 时注意溢出!

"Point.h" 19910b, 10 lines

```
template <class T>
T polygonArea2(vector<Point<T>> &v) {
    T a = v.back().cross(v[0]);

    for (int i = 0; i < (int)v.size() - 1; i++) {
        a += v[i].cross(v[i + 1]);
    }

    return a;
}
```

PolygonCenter.h

描述: 返回多边形的重心。

时间: $\mathcal{O}(n)$

"Point.h" db3724, 12 lines

```
using P = Point<double>;
P polygonCenter(const vector<P> &v) {
    P res(0, 0);
    double A = 0;

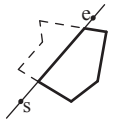
    for (int i = 0, j = (int)v.size() - 1; i < (int)v.size(); j
         = i++) {
        res = res + (v[i] + v[j]) * v[j].cross(v[i]);
        A += v[j].cross(v[i]);
    }

    return res / A / 3;
}
```

PolygonCut.h

描述:

返回一个多边形的顶点, 其中已切掉从 s 指向 e 的直线左侧的部分。



用法: vector<P> p = ...;
p = polygonCut(p, P(0,0), P(1,0));

"Point.h" 9f2abd, 15 lines

```
using P = Point<double>;
vector<P> polygonCut(const vector<P> &poly, P s, P e) {
    vector<P> res;

    for (int i = 0; i < (int)poly.size(); i++) {
        P cur = poly[i], prev = i ? poly[i - 1] : poly.back();
        auto a = s.cross(e, cur), b = s.cross(e, prev);
        if ((a < 0) != (b < 0))
            res.push_back(cur + (prev - cur) * (a / (a - b)));
        if (a < 0)
            res.push_back(cur);
    }

    return res;
}
```

ConvexHull.h

描述:

按逆时针顺序返回凸包上的点。处于凸包上另外两点之间的边上的点不计入凸包。



时间: $\mathcal{O}(n \log n)$

"Point.h" 555488, 16 lines

```
using P = Point<ll>;
vector<P> convexHull(vector<P> pts) {
    if ((int)pts.size() <= 1) return pts;
    sort(pts.begin(), pts.end());
    vector<P> h((int)pts.size() + 1);
    int s = 0, t = 0;

    for (int it = 2; it--; s = --t, reverse(pts.begin(), pts.
         end())) {
        for (P p : pts) {
            while (t >= s + 2 && h[t - 2].cross(h[t - 1], p) <=
                 0) t--;
            h[t++] = p;
        }
    }

    return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])
        };
}
```

HullDiameter.h

描述: 返回凸包 (逆时针, 无重点/共线点) 上距离最大的两点。

时间: $\mathcal{O}(n)$

"Point.h" 98a457, 15 lines

```
using P = Point<ll>;
array<P, 2> hullDiameter(vector<P> S) {
    int n = (int)S.size(), j = n < 2 ? 0 : 1;
    pair<ll, array<P, 2>> res({0, {S[0], S[0]}});

    for (int i = 0; i < j; i++) {
        for (;; j = (j + 1) % n) {
            res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}
                });
            if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i])
                >= 0)
                break;
        }
    }

    return res.second;
}
```

PointInsideHull.h

描述: 判断点 t 是否在凸包内 (逆时针顺序, 无共线点)。返回点是否在凸包内。若 strict 为真, 边界上的点不计入。

时间: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h" 12281e, 14 lines

```
using P = Point<ll>;

bool inHull(const vector<P> &l, P p, bool strict = true) {
    int a = 1, b = (int)l.size() - 1, r = !strict;
    if ((int)l.size() < 3) return r && onSegment(l[0], l.back()
        , p);
    if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
    if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p) <=
        -r)
        return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
    }
    return sgn(l[a].cross(l[b], p)) < r;
}
```

LineHullIntersection.h

描述: 直线与凸多边形的交线。多边形必须逆时针且无共线点。lineHull(line, poly) 返回描述直线与多边形交线的一对值: • $(-1,-1)$ 若不相交, • $(i,-1)$ 若只碰到顶点 i, • (i,i) 若与边 $(i,i+1)$ 重合, • (i,j) 若穿过边 $(i,i+1)$ 与 $(j,j+1)$。后一种情况中, 若穿过顶点 i, 视为发生在边 $(i,i+1)$ 上。返回的点按直线碰到多边形的顺序排列。extrVertex 返回凸包在直线方向投影最大的点。

时间: $\mathcal{O}(\log n)$

"Point.h" 45942b, 42 lines

```
#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P>
int extrVertex(vector<P> &poly, P dir) {
    int n = (int)poly.size(), lo = 0, hi = n;
    if (extr(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (extr(m)) return m;
        int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
        (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) =
            m;
    }
    return lo;
}

#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P> &poly) {
    int endA = extrVertex(poly, (a - b).perp());
    int endB = extrVertex(poly, (b - a).perp());
    if (cmpL(endA) < 0 || cmpL(endB) > 0)
        return {-1, -1};
    array<int, 2> res;

    for (int i = 0; i < 2; i++) {
        int lo = endB, hi = endA, n = (int)poly.size();
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
            (cmpL(m) == cmpL(endB) ? lo : hi) = m;
        }
        res[i] = (lo + !cmpL(hi)) % n;
        swap(endA, endB);
    }
    if (res[0] == res[1]) return {res[0], -1};
    if (!cmpL(res[0]) && !cmpL(res[1])) {
        switch ((res[0] - res[1] + (int)poly.size() + 1) % (int
            )poly.size()) {
            case 0: return {res[0], res[0]};
            case 2: return {res[1], res[1]};
        }
    }
    return res;
}
```

8.4 杂项点集问题

ClosestPair.h

描述: 求最近点对。

时间: $\mathcal{O}(n \log n)$

"Point.h" 4f2cfc, 23 lines

```
using P = Point<ll>;
pair<P, P> closest(vector<P> v) {
    assert((int)v.size() > 1);
    set<P> S;
    sort(v.begin(), v.end(), [](P a, P b) { return a.y < b.y; }
        );
    pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
    int j = 0;

    for (P p : v) {
```

```
        P d{1 + (ll)sqrt(ret.first), 0};
        while (v[j].y <= p.y - d.x) {
            S.erase(v[j++]);
        }
        auto lo = S.lower_bound(p - d), hi = S.upper_bound(p +
            d);

        for (; lo != hi; lo++) {
            ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
        }
        S.insert(p);
    }

    return ret.second;
}
```

kdTree.h

描述: KD 树 (二维, 可扩展到三维)

"Point.h" aff463, 68 lines

```
using T = long long;
using P = Point<T>;
const T INF = numeric_limits<T>::max();

bool on_x(P a, P b) { return a.x < b.x; }
bool on_y(P a, P b) { return a.y < b.y; }

struct Node {
    P pt; // 若是叶子,为其包含的唯一点
    T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // 边界
    Node *first = 0, *second = 0;

    T distance(P p) { // 到某点的最小距离平方
        T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
        T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
        return (P(x, y) - p).dist2();
    }

    Node(vector<P> &&vp) : pt(vp[0]) {
        for (P p : vp) {
            x0 = min(x0, p.x);
            x1 = max(x1, p.x);
            y0 = min(y0, p.y);
            y1 = max(y1, p.y);
        }
        if (vp.size() > 1) {
            // 若宽 >= 高则按 x 划分(并不理想……)
            sort(vp.begin(), vp.end(), x1 - x0 >= y1 - y0 ?
                on_x : on_y);
            // 每棵子树取一半数组(中点有大量重复时性能不佳)
            int half = (int)vp.size() / 2;
            first = new Node({vp.begin(), vp.begin() + half});
            second = new Node({vp.begin() + half, vp.end()});
        }
    }
};

struct KDTree {
    Node *root;

    KDTree(const vector<P> &vp) : root(new Node({vp.begin(), vp
        .end()})) {}

    pair<T, P> search(Node *node, P p) {
        if (!node->first) {
            // 若不希望找到点自身,取消下面一行的注释:
            // if (p == node->pt) return {INF, P()};
            return make_pair((p - node->pt).dist2(), node->pt);
        }

        Node *f = node->first, *s = node->second;
        T bfirst = f->distance(p), bsec = s->distance(p);
        if (bfirst > bsec) {
            swap(bsec, bfirst);
            swap(f, s);
        }

        // 先搜索较近的一侧,必要时搜索另一侧
        auto best = search(f, p);
        if (bsec < best.first)
            best = min(best, search(s, p));
        return best;
    }

    // 求到某点的最近点及其距离平方
    // (要求 Point 有任意定义的 operator<)
    pair<T, P> nearest(P p) {
        return search(root, p);
    }
};
```

FastDelaunay.h

描述: 快速 Delaunay 三角剖分。每个外接圆内不含任何输入点。不允许重复点。若所有点共线, 则不返回任何三角形。对 double 也应该可用, 但'circ' 中可能有精度问题。返回的三角形依序为 {t[0][0], t[0][1], t[0][2], t[1][0], ...}, 均逆时针。

时间: $\mathcal{O}(n \log n)$

"Point.h" 438afc, 109 lines

```
using P = Point<ll>;
using Q = struct Quad*;
using lll = __int128_t; // (坐标 < 2e4 时可以用 ll)
P arb(LLONG_MAX, LLONG_MAX); // 不等于任何其它点

struct Quad {
    Q rot, o;
    P p = arb;
    bool mark;

    P &F() { return r()->p; }
    Q &r() { return rot->rot; }
    Q prev() { return rot->o->rot; }
    Q next() { return r()->prev(); }
} *H;

bool circ(P p, P a, P b, P c) { // p 在外接圆内?
    lll p2 = p.dist2(), A = a.dist2() - p2,
        B = b.dist2() - p2, C = c.dist2() - p2;
    return p.cross(a, b) * C + p.cross(b, c) * A + p.cross(c, a
        ) * B > 0;
}

Q makeEdge(P orig, P dest) {
    Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
    H = r->o;
    r->r()->r() = r;

    for (int i = 0; i < 4; i++) {
        r = r->rot;
        r->p = arb;
        r->o = i & 1 ? r : r->r();
    }
    r->p = orig;
    r->F() = dest;
    return r;
}

void splice(Q a, Q b) {
    swap(a->o->rot->o, b->o->rot->o);
    swap(a->o, b->o);
}

Q connect(Q a, Q b) {
    Q q = makeEdge(a->F(), b->p);
    splice(q, a->next());
    splice(q->r(), b);
    return q;
}

pair<Q, Q> rec(const vector<P> &s) {
    if ((int)s.size() <= 3) {
        Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back()
            );
        if ((int)s.size() == 2) return {a, a->r()};
        splice(a->r(), b);
        auto side = s[0].cross(s[1], s[2]);
        Q c = side ? connect(b, a) : 0;
        return {side < 0 ? c->r() : a, side < 0 ? c : b->r()};
    }

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
    Q A, B, ra, rb;
    int half = (int)s.size() / 2;
    tie(ra, A) = rec({s.begin(), s.end() - half});
    tie(B, rb) = rec({s.end() - half, s.end()});

    while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
           (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
    Q base = connect(B->r(), A);
    if (A->p == ra->p) ra = base->r();
    if (B->p == rb->p) rb = base;

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
        while (circ(e->dir->F(), H(base), e->F())) { \
            Q t = e->dir; \
            splice(e, e->prev()); \
            splice(e->r(), e->r()->prev()); \
            e->o = H; H = e; e = t; \
        }
    for (;;) {
        DEL(LC, base->r(), o); DEL(RC, base, prev());
        if (!valid(LC) && !valid(RC)) break;
        if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
            base = connect(RC, base->r());
        else
            base = connect(base->r(), LC->r());
    }
    return {ra, rb};
}

vector<P> triangulate(vector<P> pts) {
    sort(pts.begin(), pts.end());
    assert(unique(pts.begin(), pts.end()) == pts.end());
    if ((int)pts.size() < 2) return {};
    Q e = rec(pts).first;
    vector<Q> q = {e};
    int qi = 0;

    while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
    q.push_back(c->r()); c = c->next(); } while (c != e); }
    ADD;
    pts.clear();

    while (qi < (int)q.size()) {
        if (!(e = q[qi++])->mark) ADD;
    }
```

```
    return pts;
}
```

8.5 三维

PolyhedronVolume.h

描述: 多面体体积的奇妙公式。面应朝外。

3058c3, 8 lines

```
template <class V, class L>
double signedPolyVolume(const V &p, const L &trilist) {
    double v = 0;

    for (auto i : trilist) v += p[i.a].cross(p[i.b]).dot(p[i.c
        ]);

    return v / 6;
}
```

Point3D.h

描述: 三维空间中的点类。T 可以取 double 或 long long。

fd0656, 37 lines

```
template <class T>
struct Point3D {
    using P = Point3D;
    T x, y, z;

    explicit Point3D(T x = 0, T y = 0, T z = 0) : x(x), y(y), z
        (z) {}
    bool operator<(P p) const {
        return tie(x, y, z) < tie(p.x, p.y, p.z);
    }
    bool operator==(P p) const {
        return tie(x, y, z) == tie(p.x, p.y, p.z);
    }
    P operator+(P p) const { return P(x + p.x, y + p.y, z + p.z
        ); }
    P operator-(P p) const { return P(x - p.x, y - p.y, z - p.z
        ); }
    P operator*(T d) const { return P(x * d, y * d, z * d); }
    P operator/(T d) const { return P(x / d, y / d, z / d); }
    T dot(P p) const { return x * p.x + y * p.y + z * p.z; }
    P cross(P p) const {
        return P(y * p.z - z * p.y, z * p.x - x * p.z, x * p.y
            - y * p.x);
    }
    T dist2() const { return x * x + y * y + z * z; }
    double dist() const { return sqrt((double)dist2()); }
    // 到 x 轴的方位角(经度),范围 [-pi, pi]
    double phi() const { return atan2(y, x); }
    // 到 z 轴的极角(纬度),范围 [0, pi]
    double theta() const { return atan2(sqrt(x * x + y * y), z)
        ; }
    P unit() const { return *this / (T)dist(); } // 使 dist()=1
    // 返回垂直于 *this 与 p 的单位向量
    P normal(P p) const { return cross(p).unit(); }

    // 返回绕轴 axis 逆时针旋转 angle 弧度后的点
    P rotate(double angle, P axis) const {
        double s = sin(angle), c = cos(angle);
        P u = axis.unit();
        return u * dot(u) * (1 - c) + (*this) * c - cross(u) *
            s;
    }
};
```

3dHull.h

描述: 计算点集三维凸包的全部面。* 任意四点都不能共面 *, 否则结果随机。所有面均朝外。

时间: $\mathcal{O}\left(n^2\right)$

"Point3D.h"

56391d, 64 lines

```
using P3 = Point3D<double>;

struct PR {
    void ins(int x) { (a == -1 ? a : b) = x; }
    void rem(int x) { (a == x ? a : b) = -1; }
    int cnt() { return (a != -1) + (b != -1); }
    int a, b;
};

struct F { P3 q; int a, b, c; };

vector<F> hull3d(const vector<P3> &A) {
    assert((int)A.size() >= 4);
    vector<vector<PR>> E((int)A.size(), vector<PR>((int)A.size
        (), {-1, -1}));
#define E(x,y) E[f.x][f.y]
    vector<F> FS;

    auto mf = [&](int i, int j, int k, int l) {
        P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
        if (q.dot(A[l]) > q.dot(A[i]))
            q = q * -1;
        F f{q, i, j, k};
        E(a, b).ins(k);
        E(a, c).ins(j);
        E(b, c).ins(i);
        FS.push_back(f);
    };

    for (int i = 0; i < 4; i++) {
        for (int j = i + 1; j < 4; j++) {
            for (int k = j + 1; k < 4; k++) {
                mf(i, j, k, 6 - i - j - k);
            }
        }
    }

    for (int i = 4; i < (int)A.size(); i++) {
        for (int j = 0; j < (int)FS.size(); j++) {
            F f = FS[j];
            if (f.q.dot(A[i]) > f.q.dot(A[f.a])) {
                E(a, b).rem(f.c);
                E(a, c).rem(f.b);
                E(b, c).rem(f.a);
                swap(FS[j--], FS.back());
                FS.pop_back();
            }
        }
        int nw = (int)FS.size();

        for (int j = 0; j < nw; j++) {
            F f = FS[j];
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
            C(a, b, c);
            C(a, c, b);
            C(b, c, a);
        }
    }
    for (F &it : FS) {
        if ((A[it.b] - A[it.a]).cross(
            A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b)
                ;
    }

    return FS;
}
```

sphericalDistance.h

描述: 返回半径为 radius 的球面上两点之间的最短距离——两点方位角 (经度, 自 x 轴起) 为 f1 (ϕ_1) 与 f2 (ϕ_2), 天顶角 (纬度, 自 z 轴起,0 = 北极) 为 t1 (θ_1) 与 t2 (θ_2)。所有角度以弧度计。算法先把球坐标转换为直角坐标, 因此若你已有直角坐标, 可以只用最后两行。dx*radius 即两点在 x 方向的差,d*radius 即两点的总距离。

611f07, 8 lines

```
double sphericalDistance(double f1, double t1,
        double f2, double t2, double radius) {
    double dx = sin(t2) * cos(f2) - sin(t1) * cos(f1);
    double dy = sin(t2) * sin(f2) - sin(t1) * sin(f1);
    double dz = cos(t2) - cos(t1);
    double d = sqrt(dx * dx + dy * dy + dz * dz);
    return radius * 2 * asin(d / 2);
}
```

字符串 (9)

KMP.h

描述: pi[x] 计算 s 中以 x 结尾的最长真前缀 (不含 s[0...x] 本身) 的长度 (abacaba -> 0010123)。可用于求一个串的全部出现位置。

时间: $\mathcal{O}\left(n\right)$

f59843, 21 lines

```
vi pi(const string &s) {
    vi p((int)s.size());

    for (int i = 1; i < (int)s.size(); i++) {
        int g = p[i - 1];
        while (g && s[i] != s[g]) g = p[g - 1];
        p[i] = g + (s[i] == s[g]);
    }

    return p;
}

vi match(const string &s, const string &pat) {
    vi p = pi(pat + '\0' + s), res;

    for (int i = (int)p.size() - (int)s.size(); i < (int)p.size
        (); i++) {
        if (p[i] == (int)pat.size()) res.push_back(i - 2 * (int
            )pat.size());
    }

    return res;
}
```

Zfunc.h

描述: z[i] 计算 s[i:] 与 s 的最长公共前缀长度, 但 z[0] = 0。(abacaba -> 0010301)

时间: $\mathcal{O}\left(n\right)$

50112d, 16 lines

```
vi Z(const string &S) {
    vi z((int)S.size());
    int l = -1, r = -1;

    for (int i = 1; i < (int)S.size(); i++) {
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
        while (i + z[i] < (int)S.size() && S[i + z[i]] == S[z[i
            ]])
            z[i]++;
        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }

    return z;
}
```

Manacher.h

描述: 对字符串的每个位置, 计算 p[0][i] = 以 i 为中心的最长偶回文的半长,p[1][i] = 最长奇回文 (半长向下取整)。
时间: $\mathcal{O}(N)$

238f05, 23 lines

```
array<vi, 2> manacher(const string &s) {
    int n = (int)s.size();
    array<vi, 2> p = {vi(n + 1), vi(n)};

    for (int z = 0; z < 2; z++) {
        for (int i = 0, l = 0, r = 0; i < n; i++) {
            int t = r - i + !z;
            if (i < r) p[z][i] = min(t, p[z][l + t]);
            int L = i - p[z][i], R = i + p[z][i] - !z;
            while (L >= 1 && R + 1 < n && s[L - 1] == s[R + 1])
                {
                p[z][i]++;
                L--;
                R++;
            }
            if (R > r) {
                l = L;
                r = R;
            }
        }
    }

    return p;
}
```

MinRotation.h

描述: 求字符串字典序最小的循环移位。
用法: rotate(v.begin(), v.begin()+minRotation(v), v.end());
时间: $\mathcal{O}(N)$

fa9063, 19 lines

```
int minRotation(string s) {
    int a = 0, N = (int)s.size();
    s += s;

    for (int b = 0; b < N; b++) {
        for (int k = 0; k < N; k++) {
            if (a + k == b || s[a + k] < s[b + k]) {
                b += max(0, k - 1);
                break;
            }
            if (s[a + k] > s[b + k]) {
                a = b;
                break;
            }
        }
    }

    return a;
}
```

SuffixArray.h

描述: 构建字符串的后缀数组。sa[i] 为排名第 i 的后缀的起始下标。返回的 vector 大小为 $n+1$, 且 sa[0] = n。lcp 数组包含后缀数组中相邻后缀的最长公共前缀: lcp[i] = lcp(sa[i], sa[i-1]),lcp[0] = 0。输入字符串不能含 nul 字符。
时间: $\mathcal{O}(n \log n)$

e1927d, 38 lines

```
struct SuffixArray {
    vi sa, lcp;

    SuffixArray(string s, int lim = 256) { // 或 vector<int>
        s.push_back(0);
        int n = (int)s.size(), k = 0, a, b;
        vi x(s.begin(), s.end()), y(n), ws(max(n, lim));
        sa = lcp = y;
        iota(sa.begin(), sa.end(), 0);

        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim =
            p) {
            p = j;
            iota(y.begin(), y.end(), n - j);

            for (int i = 0; i < n; i++) {
                if (sa[i] >= j) y[p++] = sa[i] - j;
            }
            fill(ws.begin(), ws.end(), 0);
            for (int i = 0; i < n; i++) ws[x[i]]++;
            for (int i = 1; i < lim; i++) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
            swap(x, y);
            p = 1;
            x[sa[0]] = 0;

            for (int i = 1; i < n; i++) {
                a = sa[i - 1];
                b = sa[i];
                x[b] = (y[a] == y[b] && y[a + j] == y[b + j]) ?
                      p - 1 : p++;
            }
        }

        for (int i = 0, j; i < n - 1; lcp[x[i++]] = k) {
            for (k && k--, j = sa[x[i] - 1];
                 s[i + k] == s[j + k]; k++);
        }
    }
};
```

SuffixTree.h

描述: 在线构建后缀树的 Ukkonen 算法。每个节点保存字符串中的下标区间 [l, r) 以及子节点列表。后缀由遍历该树、连接 [l, r) 子串得到。根为 0(l = -1, r = 0), 不存在的子节点为 -1。要得到完整的树, 在末尾附加一个哑符号——否则可能包含末完整的路径 (不过对子串匹配仍然有用)。
时间: $\mathcal{O}(26N)$

270594, 80 lines

```
struct SuffixTree {
    enum { N = 200010, ALPHA = 26 }; // N ~ 2*maxlen+10
    int toi(char c) { return c - 'a'; }
    string a; // v = 当前节点,q = 当前位置
    int t[N][ALPHA], l[N], r[N], p[N], s[N], v = 0, q = 0, m =
        2;

    void ukkadd(int i, int c) {
suff:
        if (r[v] <= q) {
            if (t[v][c] == -1) {
                t[v][c] = m;
                l[m] = i;
                p[m++] = v;
                v = s[v];
                q = r[v];
                goto suff;
            }
            v = t[v][c];
            q = l[v];
        }
        if (q == -1 || c == toi(a[q])) q++;
        else {
            l[m + 1] = i;
            p[m + 1] = m;
            l[m] = l[v];
            r[m] = q;
            p[m] = p[v];
            t[m][c] = m + 1;
            t[m][toi(a[q])] = v;
            l[v] = q;
            p[v] = m;
            t[p[m]][toi(a[l[m]])] = m;
            v = s[p[m]];
            q = l[m];

            while (q < r[m]) {
                v = t[v][toi(a[q])];
                q += r[v] - l[v];
            }
            if (q == r[m]) s[m] = v;
            else s[m] = m + 2;
            q = r[v] - (q - r[m]);
            m += 2;
            goto suff;
        }
    }

    SuffixTree(string a) : a(a) {
        fill(r, r + N, (int)a.size());
        memset(s, 0, sizeof s);
        memset(t, -1, sizeof t);
        fill(t[1], t[1] + ALPHA, 0);
        s[0] = 1;
        l[0] = l[1] = -1;
        r[0] = r[1] = p[0] = p[1] = 0;
        for (int i = 0; i < (int)a.size(); i++) ukkadd(i, toi(a
            [i]));
    }

    // 示例:求最长公共子串(使用 ALPHA = 28)
    pii best;
    int lcs(int node, int i1, int i2, int olen) {
        if (l[node] <= i1 && i1 < r[node]) return 1;
        if (l[node] <= i2 && i2 < r[node]) return 2;
        int mask = 0, len = node ? olen + (r[node] - l[node]) :
            0;

        for (int c = 0; c < ALPHA; c++) {
            if (t[node][c] != -1)
                mask |= lcs(t[node][c], i1, i2, len);
        }
        if (mask == 3)
            best = max(best, {len, r[node] - len});
        return mask;
    }

    static pii LCS(string s, string t) {
        SuffixTree st(s + (char)('z' + 1) + t + (char)('z' + 2)
            );
        st.lcs(0, (int)s.size(), (int)s.size() + 1 + (int)t.
            size(), 0);
        return st.best;
    }
};
```

Hashing.h

描述: 字符串哈希的若干直白方法。

ffe0e9, 61 lines

```
// 模 2^64-1 的算术。比模 2^64 慢一倍且代码更多,
// 但可以对抗恶意数据(如 Thue-Morse 串:长度 2^10 的
// ABBA... 与 BAAB... 在模 2^64 下哈希相同)。
// 如果你认为数据随机,可改用 "typedef uint64_t H;";
// 或者如果不担心生日悖论,用模 10^9+7 也可以。
using ull = unsigned long long;
struct H {
    ull x;
    H(ull x = 0) : x(x) {}
```

```
    H operator+(H o) { return x + o.x + (x + o.x < x); }
    H operator-(H o) { return *this + ~o.x; }
    H operator*(H o) {
        auto m = (__uint128_t)x * o.x;
        return H((ull)m) + (ull)(m >> 64);
    }
    ull get() const { return x + !~x; }
    bool operator==(H o) const { return get() == o.get(); }
    bool operator<(H o) const { return get() < o.get(); }
};
static const H C = (ll)1e11 + 3; // (阶约 3e9;随机也可以)

struct HashInterval {
    vector<H> ha, pw;

    HashInterval(string &str) : ha((int)str.size() + 1), pw(ha)
        {
        pw[0] = 1;

        for (int i = 0; i < (int)str.size(); i++) {
            ha[i + 1] = ha[i] * C + str[i];
            pw[i + 1] = pw[i] * C;
        }
    }

    H hashInterval(int a, int b) { // 哈希 [a, b)
        return ha[b] - ha[a] * pw[b - a];
    }
};

vector<H> getHashes(string &str, int length) {
    if ((int)str.size() < length) return {};
    H h = 0, pw = 1;

    for (int i = 0; i < length; i++) {
        h = h * C + str[i];
        pw = pw * C;
    }
    vector<H> ret = {h};

    for (int i = length; i < (int)str.size(); i++) {
        ret.push_back(h = h * C + str[i] - pw * str[i - length
            ]);
    }

    return ret;
}

H hashString(string &s) {
    H h{};

    for (char c : s) h = h * C + c;
    return h;
}
```

AhoCorasick.h

描述: Aho-Corasick 自动机, 用于多模式匹配。用 AhoCorasick ac(patterns) 初始化; 自动机起点为下标 0。find(word) 对每个位置返回以该位置结尾的最长单词的下标, 没有则为 -1。findAll(−, word) 求出每个位置开始的所有单词 (若无重复模式, 至多 $N\sqrt{N}$ 个) (短的在前)。允许重复模式; 不允许空模式。要求每个位置开始的最长单词, 把所有输入反转即可。对大字符集, 把每个符号拆成若干块, 块间用哨兵位分隔。
时间: 构建 $\mathcal{O}(26N)$,N 为模式长度之和。find(x) 为 $\mathcal{O}(N)$,N 为 x 的长度。findAll 为 $\mathcal{O}(NM)$。

d043f2, 80 lines

```
struct AhoCorasick {
    enum {alpha = 26, first = 'A'}; // 修改这里!
    struct Node {
        // (nmatches 可选)
        int back, next[alpha], start = -1, end = -1, nmatches =
            0;

        Node(int v) { memset(next, v, sizeof(next)); }
    };

    vector<Node> N;
    vi backp;

    void insert(string &s, int j) {
        assert(!s.empty());
        int n = 0;

        for (char c : s) {
            int &m = N[n].next[c - first];
            if (m == -1) {
                n = m = (int)N.size();
                N.emplace_back(-1);
            }
            else n = m;
        }
        if (N[n].end == -1) N[n].start = j;
        backp.push_back(N[n].end);
        N[n].end = j;
        N[n].nmatches++;
    }

    AhoCorasick(vector<string> &pat) : N(1, -1) {
        for (int i = 0; i < (int)pat.size(); i++) insert(pat[i
            ], i);
        N[0].back = (int)N.size();
        N.emplace_back(0);

        queue<int> q;
        for (q.push(0); !q.empty(); q.pop()) {
            int n = q.front(), prev = N[n].back;
            for (int i = 0; i < alpha; i++) {
                int &ed = N[n].next[i], y = N[prev].next[i];
                if (ed == -1) ed = y;
                else {
                    N[ed].back = y;
                    (N[ed].end == -1 ? N[ed].end : backp[N[ed].
                        start])
                        = N[y].end;
                    N[ed].nmatches += N[y].nmatches;
                    q.push(ed);
                }
            }
        }
    }

    vi find(string word) {
        int n = 0;
        vi res; // ll count = 0;

        for (char c : word) {
            n = N[n].next[c - first];
            res.push_back(N[n].end);
            // count += N[n].nmatches;
        }

        return res;
    }

    vector<vi> findAll(vector<string> &pat, string word) {
        vi r = find(word);
        vector<vi> res((int)word.size());

        for (int i = 0; i < (int)word.size(); i++) {
            int ind = r[i];
            while (ind != -1) {
                res[i - (int)pat[ind].size() + 1].push_back(ind
                    );
                ind = backp[ind];
            }
        }

        return res;
    }
};
```

杂项 (10)

10.1 区间

IntervalContainer.h

描述: 在一组不相交区间上增删区间。添加时会与被覆盖的区间合并。区间为 [左闭, 右开)。
时间: $\mathcal{O}(\log N)$

edce47, 26 lines

```
set<pii>::iterator addInterval(set<pii> &is, int L, int R) {
    if (L == R) return is.end();
    auto it = is.lower_bound({L, R}), before = it;

    while (it != is.end() && it->first <= R) {
        R = max(R, it->second);
        before = it = is.erase(it);
    }
    if (it != is.begin() && (--it)->second >= L) {
        L = min(L, it->first);
        R = max(R, it->second);
        is.erase(it);
    }

    return is.insert(before, {L, R});
}

void removeInterval(set<pii> &is, int L, int R) {
    if (L == R) return;
    auto it = addInterval(is, L, R);
    auto r2 = it->second;

    if (it->first == L) is.erase(it);
    else (int &)it->second = L;
    if (R != r2) is.emplace(R, r2);
}
```

IntervalCover.h

描述: 计算覆盖给定区间的最小若干区间的下标。区间应为 [左闭, 右开)。要支持 [左闭, 右闭], 把 (A) 处改为加上 || R.empty()。失败 (或 G 为空) 时返回空集。
时间: $\mathcal{O}(N \log N)$

3bfc85, 22 lines

```
template <class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I) {
    vi S((int)I.size()), R;
    iota(S.begin(), S.end(), 0);
    sort(S.begin(), S.end(), [&](int a, int b) { return I[a] <
        I[b]; });
    T cur = G.first;
    int at = 0;

    while (cur < G.second) { // (A)
        pair<T, int> mx = make_pair(cur, -1);

        while (at < (int)I.size() && I[S[at]].first <= cur) {
            mx = max(mx, make_pair(I[S[at]].second, S[at]));
            at++;
```

```
        }
        if (mx.second == -1) return {};
        cur = mx.first;
        R.push_back(mx.second);
    }

    return R;
}
```

ConstantIntervals.h

描述: 把 [from, to) 上的单调函数切分为最少的、取相同值的半开区间。对每个区间运行一次回调 g。

用法: constantIntervals(0, sz(v), [&](int x){return v[x];}, [&](int lo, int hi, T val){...});

时间: $\mathcal{O}\left(k \log \frac{n}{k}\right)$

753a4c, 22 lines

```
template <class F, class G, class T>
void rec(int from, int to, F &f, G &g, int &i, T &p, T q) {
    if (p == q) return;
    if (from == to) {
        g(i, to, p);
        i = to;
        p = q;
    } else {
        int mid = (from + to) >> 1;
        rec(from, mid, f, g, i, p, f(mid));
        rec(mid + 1, to, f, g, i, p, q);
    }
}

template <class F, class G>
void constantIntervals(int from, int to, F f, G g) {
    if (to <= from) return;
    int i = from;
    auto p = f(i), q = f(to - 1);
    rec(from, to - 1, f, g, i, p, q);
    g(i, to, q);
}
```

10.2 杂项算法

TernarySearch.h

描述: 求 $[a, b]$ 中使 $f(i)$ 最大的最小 i, 假设 $f(a) < \ldots < f(i) \geq \cdots \geq f(b)$。要反转哪一侧允许非严格不等式, 把标有 (A) 的 < 改成 <=, 并反转 (B) 处的循环。要求最小值, 把 (B) 处的 < 改成 >。

用法: int ind = ternSearch(0,n-1,[&](int i){return a[i];});

时间: $\mathcal{O}\left(\log(b-a)\right)$

a14abb, 13 lines

```
template <class F>
int ternSearch(int a, int b, F f) {
    assert(a <= b);
    while (b - a >= 5) {
        int mid = (a + b) / 2;
        if (f(mid) < f(mid + 1)) a = mid; // (A)
        else b = mid + 1;
    }
    for (int i = a + 1; i < b + 1; i++) {
        if (f(a) < f(i)) a = i; // (B)
    }
    return a;
}
```

LIS.h

描述: 计算最长上升子序列的下标。

时间: $\mathcal{O}\left(N \log N\right)$

6e8d11, 27 lines

```
template <class I>
vi lis(const vector<I> &S) {
    if (S.empty()) return {};
    vi prev((int)S.size());
    using p = pair<I, int>;
    vector<p> res;

    for (int i = 0; i < (int)S.size(); i++) {
        // 改成 i 即为最长不降子序列
        auto it = lower_bound(res.begin(), res.end(), p{S[i], 0
            });
        if (it == res.end()) {
            res.emplace_back();
            it = res.end() - 1;
        }
        *it = {S[i], i};
        prev[i] = it == res.begin() ? 0 : (it - 1)->second;
    }
    int L = (int)res.size(), cur = res.back().second;
    vi ans(L);

    while (L--) {
        ans[L] = cur;
        cur = prev[cur];
    }

    return ans;
}
```

FastKnapsack.h

描述: 给定 N 个非负整数权值 w 与非负目标 t, 计算可作为某些权值子集和的最大的 S <= t。

时间: $\mathcal{O}\left(N \max(w_i)\right)$

797319, 22 lines

```
int knapsack(vi w, int t) {
    int a = 0, b = 0, x;

    while (b < (int)w.size() && a + w[b] <= t) a += w[b++];
    if (b == (int)w.size()) return a;
    int m = *max_element(w.begin(), w.end());
    vi u, v(2 * m, -1);
    v[a + m - t] = b;

    for (int i = b; i < (int)w.size(); i++) {
        u = v;

        for (x = 0; x < m; x++) v[x + w[i]] = max(v[x + w[i]],
            u[x]);
        for (x = 2 * m; --x > m;) {
            for (int j = max(0, u[x]); j < v[x]; j++) {
                v[x - w[j]] = max(v[x - w[j]], j);
            }
        }
    }
    for (a = t; v[a + m - t] < 0; a--);
    return a;
}
```

10.3 动态规划

KnuthDP.h

描述: 区间 DP 时:$a[i][j] = \min_{i<k<j}(a[i][k] + a[k][j]) + f(i, j)$, 其中 (最小的) 最优 k 随 i 与 j 递增, 则可以按区间长度递增的顺序求解, 并把 $a[i][j]$ 的 $k = p[i][j]$ 的搜索范围限制在 $p[i][j-1]$ 与 $p[i+1][j]$ 之间。这就是 Knuth DP。充分条件是: 对所有 $a \leq b \leq c \leq d$ 有 $f(b, c) \leq f(a, d)$ 且 $f(a, c) + f(b, d) \leq f(a, d) + f(b, c)$。另可参见:LineContainer(数据结构章)、单调队列、三分搜索。

时间: $\mathcal{O}\left(N^2\right)$

DivideAndConquerDP.h

描述: 给定 $a[i] = \min_{lo(i)\leq k<hi(i)}(f(i, k))$, 其中 (最小的) 最优 k 随 i 递增, 计算 $i = L..R-1$ 的 $a[i]$。

时间: $\mathcal{O}\left((N + (hi - lo))\log N\right)$

ec5ba8, 21 lines

```
struct DP { // 随意修改:
    int lo(int ind) { return 0; }
    int hi(int ind) { return ind; }
    ll f(int ind, int k) { return dp[ind][k]; }
    void store(int ind, int k, ll v) { res[ind] = pii(k, v); }

    void rec(int L, int R, int LO, int HI) {
        if (L >= R) return;
        int mid = (L + R) >> 1;
        pair<ll, int> best(LLONG_MAX, LO);

        for (int k = max(LO, lo(mid)); k < min(HI, hi(mid)); k
            ++) {
            best = min(best, make_pair(f(mid, k), k));
        }
        store(mid, best.second, best.first);
        rec(L, mid, LO, best.second + 1);
        rec(mid + 1, R, best.second, HI);
    }

    void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};
```

10.4 调试技巧

- signal(SIGSEGV, [](int) { _Exit(0); }); 把段错误变成答案错误。类似地可以捕获 SIGABRT(断言失败) 与 SIGFPE(除以零)。_GLIBCXX_DEBUG 失败会产生 SIGABRT(gcc 5.4.0 上似乎是 SIGSEGV)。
- feenableexcept(29); 遇到 NaN(1)、除零 (4)、无穷 (8) 与非规格化数 (16) 时终止程序。

10.5 优化技巧

__builtin_ia32_ldmxcsr(40896); 禁用非规格化数 (浮点数接近最小值时会慢 20 倍)。

10.5.1 位运算技巧

- x & -x 是 x 的最低位。
- for (int x = m; x;) { --x &= m; ... } 遍历 m 的所有子集掩码 (不含 m 本身)。
- c = x&-x, r = x+c; (((r^x) >> 2)/c) | r 是 x 之后下一个 1 的个数相同的数。
- rep(b,0,K) rep(i,0,(1 << K))
  if (i & 1 << b) D[i] += D[i^(1 << b)]; 计算所有子集和。

10.5.2 Pragma

- #pragma GCC optimize ("Ofast") 会让 GCC 自动向量化循环, 并更好地优化浮点。
- #pragma GCC target ("avx2") 可以让向量化代码快一倍, 但在老机器上会崩溃。
- #pragma GCC optimize ("trapv") 遇到整数溢出时终止程序 (但非常慢)。

FastMod.h

描述: 计算 $a\%b$ 比常规快约 5 倍, 其中 b 是常量但编译期未知。返回与 $a \pmod b$ 同余、且在 $[0, 2b)$ 内的值。

e79cd0, 9 lines

```
using ull = unsigned long long;
struct FastMod {
    ull b, m;

    FastMod(ull b) : b(b), m(-1ULL / b) {}
    ull reduce(ull a) { // a % b + (0 或 b)
        return a - (ull)((__uint128_t(m) * a) >> 64) * b;
    }
};
```

FastInput.h

描述: 从 stdin 读取一个整数。用法要求程序从文件重定向输入。
用法: ./a.out < input.txt
时间: 约比 cin/scanf 快 5 倍。

26bae6, 18 lines

```
inline char gc() { // 类似 getchar()
    static char buf[1 << 16];
    static size_t bc, be;
    if (bc >= be) {
        buf[0] = 0;
        bc = 0;
        be = fread(buf, 1, sizeof(buf), stdin);
    }
    return buf[bc++]; // EOF 时返回 0
}

int readInt() {
    int a, c;
    while ((a = gc()) < 40);
    if (a == '-') return -readInt();
    while ((c = gc()) >= 48) a = a * 10 + c - 480;
    return a - 48;
}
```

BumpAllocator.h

描述: 需要动态分配很多对象且不在意释放时使用。否则"new X" 的开销约为每次分配 0.05us + 16 字节。

745db2, 8 lines

```
// 全局定义,或放在单个类里:
static char buf[450 << 20];
void *operator new(size_t s) {
    static size_t i = sizeof buf;
    assert(s < i);
    return (void *)&buf[i -= s];
}
void operator delete(void *) {}
```

SmallPtr.h

描述: 指向 BumpAllocator 内存的 32 位指针。

"BumpAllocator.h" 2dd6c9, 12 lines

```
template <class T>
struct ptr {
    unsigned ind;

    ptr(T *p = 0) : ind(p ? unsigned((char *)p - buf) : 0) {
        assert(ind < sizeof buf);
    }
    T &operator*() const { return *(T *)(buf + ind); }
    T *operator->() const { return &**this; }
    T &operator[](int a) const { return (&**this)[a]; }
    explicit operator bool() const { return ind; }
};
```

BumpAllocatorSTL.h

描述: 用于 STL 容器的 BumpAllocator。
用法: vector<vector<int, small<int>>> ed(N);

a64201, 17 lines

```
char buf[450 << 20] alignas(16);
size_t buf_ind = sizeof buf;

template <class T>
struct small {
    using value_type = T;

    small() {}
    template <class U>
    small(const U &) {}
    T *allocate(size_t n) {
        buf_ind -= n * sizeof(T);
        buf_ind &= 0 - alignof(T);
        return (T *)(buf + buf_ind);
    }
    void deallocate(T *, size_t) {}
};
```

SIMD.h

描述: SSE/AVX 内联函数的速查表, 用于同时对多个数做运算。可带来约 4 倍的常数级提升, 与循环展开正交。操作遵循模式"_mm(256)?_name_(si(128|256)|epi(8|16|32|64)|pd|ps)"。这里未全部列出; 更多内容可对 __mm__ 检索 /usr/lib/gcc/*/4.9/include/。若不支持 AVX, 可尝试 128 位操作、"emmintrin.h", 并在包含前 #define ___SSE___ 与 ___MMX___。对齐内存请用 _mm_malloc(size, 32) 或 int buf[N] alignas(32), 但更推荐 loadu/storeu。

3c7e1a, 56 lines

```
#pragma GCC target ("avx2") // 或 sse4.1
#include "immintrin.h"

using mi = __m256i;
#define L(x) _mm256_loadu_si256((mi*)&(x))

// 高层/专用方法:
// load(u)?_si256, store(u)?_si256, setzero_si256, _mm_malloc
// blendv_(epi8|ps|pd) (z?y:x), movemask_epi8 (hibits of bytes)
// i32gather_epi32(addr, x, 4): map addr[] over 32-b parts of x
// sad_epu8: sum of absolute differences of u8, outputs 4xi64
// maddubs_epi16: dot product of unsigned i7's, outputs 16xi15
// madd_epi16: dot product of signed i16's, outputs 8xi32
// extractf128_si256(, i) (256->128), cvtsi128_si32 (128->lo32)
// permute2f128_si256(x,x,1) swaps 128-bit lanes
// shuffle_epi32(x, 3*64+2*16+1*4+0) == x for each lane
// shuffle_epi8(x, y) takes a vector instead of an imm

// 适用于大多数数据类型的方法(如后缀 _epi32):
// set1, blend (i8?x:y), add, adds (sat.), mullo, sub, and/or,
// andnot, abs, min, max, sign(1,x), cmp(gt|eq), unpack(lo|hi)

int sumi32(mi m) {
    union {int v[8]; mi m;} u;
    u.m = m;
    int ret = 0;
    for (int i = 0; i < 8; i++) ret += u.v[i];
    return ret;
}
mi zero() { return _mm256_setzero_si256(); }
mi one() { return _mm256_set1_epi32(-1); }
bool all_zero(mi m) { return _mm256_testz_si256(m, m); }
bool all_one(mi m) { return _mm256_testc_si256(m, one()); }

ll example_filteredDotProduct(int n, short *a, short *b) {
    int i = 0;
    ll r = 0;
    mi zero = _mm256_setzero_si256(), acc = zero;
    while (i + 16 <= n) {
        mi va = L(a[i]), vb = L(b[i]);
        i += 16;
        va = _mm256_and_si256(_mm256_cmpgt_epi16(vb, va), va);
        mi vp = _mm256_madd_epi16(va, vb);
        acc = _mm256_add_epi64(_mm256_unpacklo_epi32(vp, zero),
            _mm256_add_epi64(acc, _mm256_unpackhi_epi32(vp,
                zero)));
    }
    union {ll v[4]; mi m;} u;
    u.m = acc;
    for (int i = 0; i < 4; i++) r += u.v[i];
    for (; i < n; i++) {
        if (a[i] < b[i]) r += a[i] * b[i]; // <- 等价
    }

    return r;
}
```

技巧 (A)

techniques.txt 159 lines

- 递归
- 分治
  - 以 N log N 找有趣的点
- 算法分析
  - 主定理
  - 均摊时间复杂度
- 贪心算法
  - 调度
  - 最大连续子段和
  - 不变量
  - Huffman 编码
- 图论
  - 动态图(额外记账)
  - 广度优先搜索
  - 深度优先搜索
  - * 普通树 / DFS 树
  - Dijkstra 算法
  - MST: Prim 算法
  - Bellman-Ford
  - König 定理与点覆盖
  - 最小费用最大流
  - Lovász 翻转
  - 矩阵树定理
  - 最大匹配, 一般图
  - Hopcroft-Karp
  - Hall 婚姻定理
  - 可图序列
  - Floyd-Warshall
  - 欧拉回路
  - 流网络
  - * 增广路
  - * Edmonds-Karp
  - 二分图匹配
  - 最小路径覆盖
  - 拓扑排序
  - 强连通分量
  - 2-SAT
  - 割点、割边与点双连通分量
  - 边染色
  - * 树
  - 点染色
  - * 二分图(=> 树)
  - * 3^n(集合覆盖的特例)
  - 直径与重心
  - 第 K 短路
  - 最短环
- 动态规划
  - 背包
  - 找零
  - 最长公共子序列
  - 最长上升子序列
  - dag 上的路径数
  - dag 上的最短路
  - 区间 DP
  - 子集 DP
  - 概率 DP
  - 树上 DP
  - 3^n 集合覆盖
  - 分治
  - Knuth 优化
  - 凸包优化
  - RMQ(稀疏表, 即 2^k 倍增)
  - 双调回路
  - 对数划分(遍历最受限的)
- 组合数学
  - 二项式系数的计算
  - 鸽巢原理
  - 容斥
  - Catalan 数
  - Pick 定理
- 数论
  - 整数部分
  - 整除
  - 欧几里得算法
  - 模运算
  - * 模乘
  - * 模逆元
  - * 快速幂取模
  - 中国剩余定理
  - 费马小定理
  - 欧拉定理
  - Phi 函数
  - Frobenius 数
  - 二次互反律
  - Pollard-Rho
  - Miller-Rabin
  - Hensel 提升
  - Vieta 跳根
- 博弈论
  - 组合博弈
  - 博弈树
  - 极小极大
  - Nim
  - 图上的游戏
  - 带环图上的游戏
  - Grundy 数
  - 无重复的二分图游戏
  - 无重复的一般游戏
  - Alpha-beta 剪枝
- 概率论
- 最优化
  - 二分
  - 三分
  - 单峰性与凸函数
  - 对导数二分
- 数值方法
  - 数值积分
  - 牛顿法
  - 二分/三分找根
  - 黄金分割搜索
- 矩阵
  - 高斯消元
  - 快速幂
- 排序
  - 基数排序
- 计算几何
  - 坐标与向量
  - * 叉积
  - * 点积
  - 凸包
  - 多边形切割
  - 最近点对
  - 坐标压缩
  - 四叉树
  - KD 树
  - 所有线段-线段交点
- 扫描线
  - 离散化(转成事件并扫描)
  - 角度扫描
  - 直线扫描
  - 离散二阶导数
- 字符串
  - 最长公共子串
  - 回文子序列
  - Knuth-Morris-Pratt
  - Trie
  - 滚动多项式哈希
  - 后缀数组
  - 后缀树
  - Aho-Corasick
  - Manacher 算法
  - 字母位置表
- 组合搜索
  - 折半枚举
  - 剪枝暴力
  - 最佳优先(A*)
  - 双向搜索
  - 迭代加深 DFS / A*
- 数据结构
  - LCA(树上 2^k 倍增的推广)
  - 树上的 pull/push 技术
  - 树链剖分
  - 点分治
  - 懒标记下传
  - 平衡树
  - 凸包技巧(convex hull trick)
  - 单调队列 / 单调栈 / 滑动队列
  - 用两个栈实现滑动队列
  - 可持久化线段树